*“The city of Salzburg as a sketch” als Prompt zu to **DALL-E Mini / OpenAI DALL-E***
https://huggingface.co/spaces/dalle-mini/dalle-mini

AI Eingangswerkstatt

(bzw. **Einführung in Artificial Intelligence** im BA Informatik)

Christine Bauer & **Roland Kwitt** (VO), Wolfgang Trutschnig (UE)

Wintersemester 2026/2027 (LV Nr.: 536.101, 511.061)

Herzlich Willkommen zum Bachelorstudium Artificial Intelligence

Übersicht

- Administratives zur Lehrveranstaltung
- Aufbau/Ablauf des Studiums
- Studienrecht
- Struktur/Organisation der Universität

- Was ist eigentlich künstliche Intelligenz?
- Grundlagen und Teilgebiete
- Intelligente Agenten
- usw. (wird laufend erweitert)

Administratives zur Lehrveranstaltung

Administratives zur Lehrveranstaltung (LV)

Univ.-Prof. Dr. Christine Bauer
Professor für
Interactive Intelligent Systems

Univ.-Prof. Dr. Roland Kwitt
Professor für
Maschinelles Lernen

Vorlesungsteil (VO) der Lehrveranstaltung

Univ.-Prof. Dr. Wolfgang Trutschnig
Professor für
Statistik / Stochastik
Direktor IDA Lab Salzburg

Übungsteil (UE) der Lehrveranstaltung

Administratives zur Lehrveranstaltung (LV)

- **Abhaltung**:
  - Vorlesung (VO, 536.101, 511.061): Mittwochs **13:00-14:30** (Jakob-Haringer-Str. 2A, **HS2 Liese Meitner**, JAK2AUG.069)
  - Übung (UE, 536.102): Donnerstags **16:00-17:45** (Hellbrunnerstrasse 34, **HS421**, H34OG2.D-041)
- **Benotung (VO Teil)**: Prüfung am Ende des Semesters (drei angebotene Termine)
- Die VO ist Teil der **STEOP** (Studieneingangs- und orientierungsphase im BA AI)
- **Unterlagen** (auch in PLUSonline, siehe QR Code): https://github.com/rkwitt/teaching

Die Unterlagen zur Lehrveranstaltung (VO) sind die **Folien**.

Administratives zur Lehrveranstaltung (LV)

Anmerkungen zu den Folien

Auf den Folien sind Referenzen/Quellen meist in der Form (Autor(en), Jahr) angegeben; detaillierte Referenzen finden Sie als Fußnoten. Nahezu alle Referenzen sind im Internet frei verfügbar.

Administratives zur Lehrveranstaltung (LV)

Stellung im **BA Informatik**

Die LV (**Einführung in Artificial Intelligence** , im Ausmaß von 1 SSt.) ist im Modul **P2**, siehe Curriculum BA Informatik 2022, verankert.

Das reduzierte Stundenausmaß äußert sich so, dass die LV frühzeitig während des Semesters endet (und natürlich nur jene bis dahin besprochenen Inhalte prüfungsrelevant sind).

Sie sind natürlich herzlich dazu eingeladen, die ganze LV zu besuchen :)

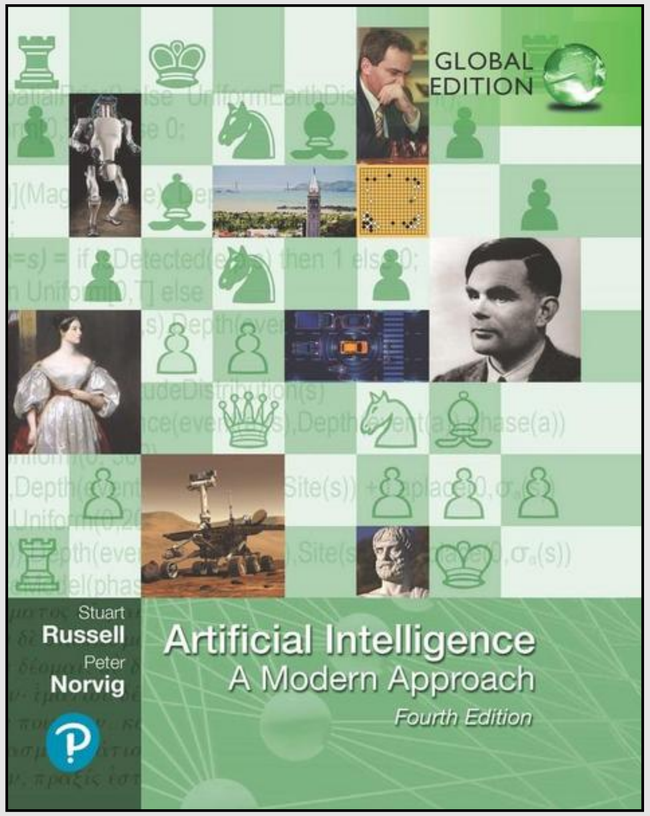
Administratives zur Lehrveranstaltung

Empfohlene Literatur (Hauptlehrbuch zur LV)

Stuart Russel & Peter Norvig

Artificial Intelligence - A Modern Approach (4th ed.)

Pearson Education Limited (2021)



Es gibt auch eine deutschsprachige Version; wir empfehlen jedoch die 2021 erschienene Originalversion.

Auf den Folien als (RN) abgekürzt!

Aufbau / Ablauf des Studiums / Code of Conduct

Aufbau / Ablauf des Studiums

Einige Grunddaten

- **Bachelorstudium** (Abschluss mit **BSc**)
- Studiendauer (Regelstudiendauer): **6 Semester**
- Studenausmaß: **180 ECTS**
- **Hauptinformationsquelle**: Curriculum

Ansprechperson(en):

- Roland Kwitt (bei fachlichen Anliegen),
- Petra Kirchweger (administrative Anliegen)

  Jakob-Haringer-Str. 2, Itzling

.

Aufbau / Ablauf des Studiums

STEOP

- Studieneingangs- und -orientierungsphase
- Die STEOP vermittelt einen Überblick über wesentliche Inhalte des Studiums
- **muss bestanden werden**, um weitere LV abschließen zu können!

  → **Ausnahme**: LV im Ausmaß von 22 ECTS können vorgezogen werden

- Im Bachelorstudium Artificial Intelligence besteht die STEOP* aus:

  AI Eingangswerkstatt (VO, 2 ECTS)

  Einführung in die Programmierung (VO, 3 ECTS)

  Grundlagen der Mathematik (VU, 3 ECTS)

** die STEOP LV bei Studieneinstieg im **Sommersemester** sind unterschiedlich; siehe Curriculum*

Aufbau / Ablauf des Studiums

LV Typen

- **Vorlesung (VO)**: Präsentation der Inhalte, Prüfungstermine nach Beendigung der LV
- **Übung (UE)**: Anwesenheitspflicht, Tests, Aufgaben vorführen / besprechen, Vorträge
- **Proseminar (PS)**: Anwesenheitspflicht, Tests, Aufgaben vorführen / besprechen, Vorträge
- **Übung mit Vorlesung (UV)**: Mischung aus VO+UE, Anwesenheitspflicht
- **Seminar (SE)**: Anwesenheitspflicht, eingehende Bearbeitung eines Themas mittels Vorträgen und Diskussionen

Eine genaue(re) Beschreibung der verschiedenen LV Typen finden sie im Curriculum.

Code of Conduct

Grundregel: Gegenseitiges respektvolles und professionelles Miteinander.

Code of Conduct (2025)

Code of Conduct (Beispiele)

- Prinzipiell Vortragende über die offizielle vorname.nachname@plus.ac.at Email-Adresse kontaktieren.
- Emails immer mit **klarem Subject/Betreff** und **professionellem Umgangston** (z.B. per Sie).

Code of Conduct (Beispiele)

Beispiel (OK)

To: Kwitt Roland <roland.kwitt@plus.ac.at>
Subject: Anrechnungen [anonymized]

Sehr geehrter Herr Prof. Kwitt,

ich bitte Sie um Genehmigung meines Antrags zur [anonymized] in der [anonymized] der letzten beiden Semester als Freie Wahlfächer. Es handelt sich dabei, wie im Anhang zu sehen, um die Tätigkeit als [anonymized] im … (2 x 6 ECTS).

Meine Daten:
Name: [anonymized]
Matr. Nr.: [anonymized]
Studium: Bachelor Artificial Intelligence

Vielen Dank im Voraus.
Freundliche Grüße
[anonymized]

Code of Conduct (Beispiele)

Beispiel (nicht OK)

To: Kwitt Roland <roland.kwitt@plus.ac.at>
Subject: Dringend

Lieber Roland,

könntest du mir so schnell wie möglich meinen Anrechnungsantrag für die VO unterschreiben.

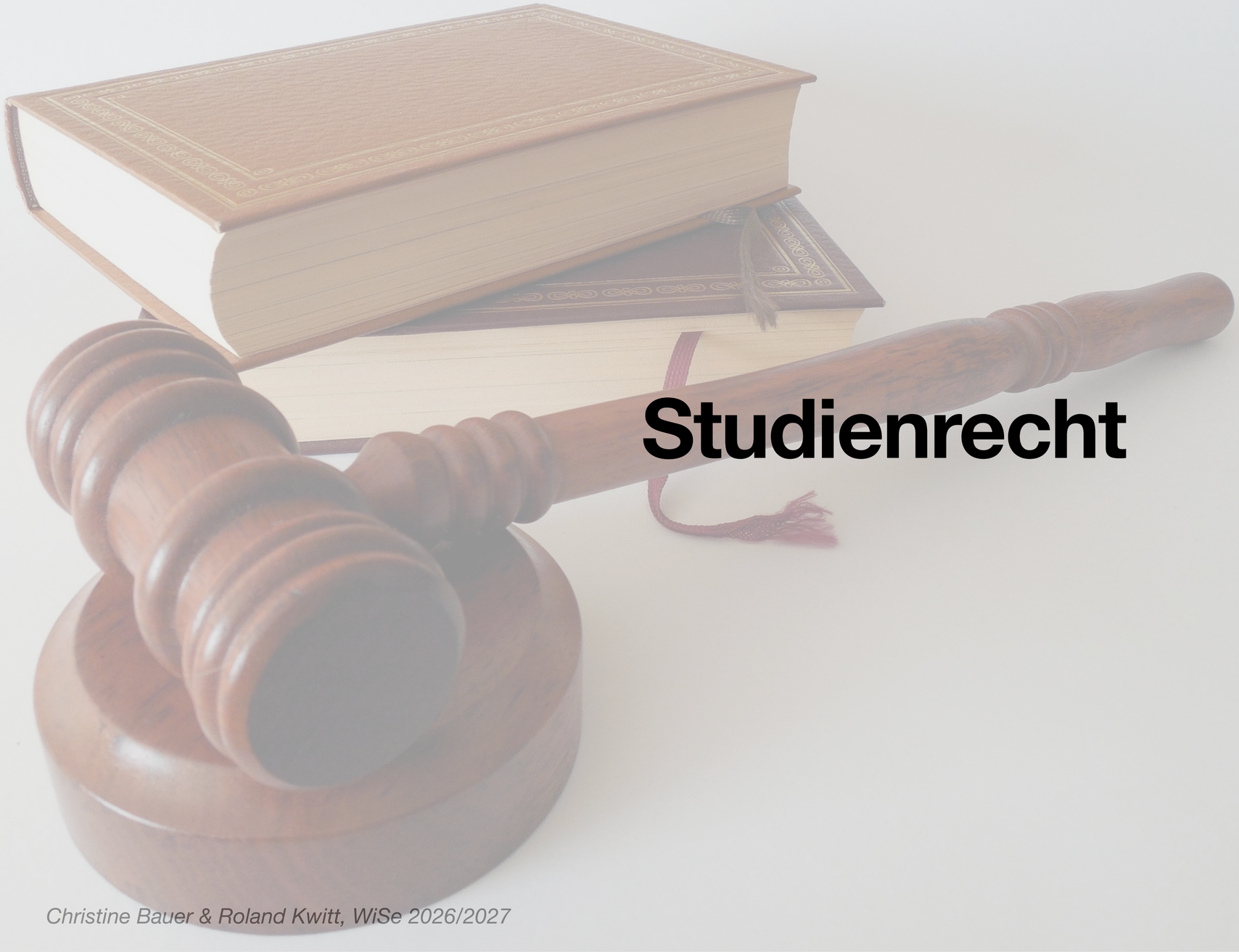
Lg,
[anonymized]

Code of Conduct (Beispiele)

Beispiel (nicht OK)

Code of Conduct (Beispiele)

Beispiel (nicht OK)



Studienrecht

Studienrecht

Die folgenden Angaben beziehen sich auf das Universitätsgesetz 2002 (kurz UG 02, in der aktuellen Fassung) und fassen einige (mMn) relevante Punkte zusammen.

Wir empfehlen die entsprechend Paragraphen im UG 02 zu lesen, da etwaige Nuancen aufgrund der verkürzten Darstellung möglicherweise nicht juristischen Maßstäben entsprechen (können).

Studienrecht

Rechte und Pflichten der Studierenden

In **§ 59 (1)** heißt es *„Den Studierenden steht nach Maßgabe der gesetzlichen Bestimmungen Lernfreiheit zu.“* Dies bedeutet unter anderem:

- Nach Maßgabe des Lehrangebotes und nach Maßgabe der Curricula kann aus Lehrveranstaltungen ausgewählt werden.
- Nach Maßgabe der universitären Regelungen kann Thema und Betreuer*in der Bachelorarbeit vorgeschlagen werden oder aus einer Anzahl von Vorschlägen ausgewählt.

Studienrecht

Rechte und Pflichten der Studierenden

Bitte überprüfen Sie **regelmäßig** Ihre Studierenden-Email!

Über die Pflichten gibt **§ 59 (2)** Auskunft:

- Namens- und Adressenänderungen sind unverzüglich bekanntzugeben.
- Die Fortsetzung des Studiums ist jedes Semester während der allgemeinen Zulassungsfrist (oder der Nachfrist) zu melden.
- Bei vorhersehbarer Studieninaktivität ist eine zeitgerechte Abmeldung vom Studium durchzuführen.
- Fristgerechte An- und Abmeldung zu den Prüfungen; nicht erscheinen ohne ordnungsgemäße Abmeldung: Ablegung frühestens nach 40 Kalendertagen möglich (Satzung der Universität Salzburg, **§ 15**).

Studienrecht

Erlöschen der Zulassung

Die wesentlichen Gründe, damit die Zulassung erlischt sind (siehe **§ 68**):

- Abmeldung vom Studium
- Die Meldung der Fortsetzung des Studiums unterbleibt, ohne beurlaubt zu sein.
- Die letzte zulässige Wiederholung einer vorgeschriebenen Prüfung wird negativ beurteilt.
- Das Studium wurde durch die positive Beurteilung bei der letzten vorgeschriebenen Prüfung abgeschlossen.

Studienrecht

Feststellung des Studienerfolges (1)

Generell heißt es dazu im **§ 72**: „Der Studienerfolg ist durch die Prüfungen und die Beurteilung wissenschaftlicher Arbeiten (Master-, Diplomarbeiten und Dissertationen) festzustellen.“

Der positive Erfolg von Prüfungen und wissenschaftlichen Arbeiten ist mit „sehr gut“ (1), „gut“ (2), „befriedigend“ (3) oder „genügend“ (4), der negative Erfolg ist mit „nicht genügend“ (5) zu beurteilen …
wenn diese Form der Beurteilung bei Lehrveranstaltungsprüfungen unmöglich oder unzweckmäßig ist, hat die positive Beurteilung „mit Erfolg teilgenommen“, die negative Beurteilung „ohne Erfolg teilgenommen“ zu lauten.

Studienrecht

Feststellung des Studienerfolges (2)

Zur Ablegung einer Prüfung ist eine **Anmeldung erforderlich** (Internet, LV-Leitung, …). Die entsprechenden Zeugnisse sind bis spätestens **vier Wochen** nach Erbringung der zu beurteilenden Leistung auszustellen.

Anmerkung: In der Realität dauert dies oft länger. Sollte ein Studienabschluss durch fehlende Noten blockiert sein, **melden Sie sich zwecks Nachfrage direkt bei den Lehrenden**.

Studienrecht

Feststellung des Studienerfolges (3)

Prüfungen sind für **nichtig** zu erklären (**§ 73**) …

- … wenn die Anmeldung zur Prüfung erschlichen wurde.
- … bei schwerwiegendem wissenschaftlichem Fehlverhalten – insbesondere durch unerlaubte Hilfsmittel, unerlaubter Weise einer anderen Person bedienen, Daten/Ergebnisse erfunden/gefälscht, Plagiat,…
- Die Prüfung, deren Beurteilung für nichtig erklärt wurde, ist auf die Gesamtanzahl der Wiederholungen anzurechnen.

- Prüfungen, die außerhalb des Wirkungsbereichs einer Fortsetzungsmeldung abgelegt wurden, sind absolut nichtig.

Studienrecht

Wiederholung von Prüfungen (1)

Positiv beurteilte Prüfungen können bis zwölf Monate nach der Ablegung einmal wiederholt werden. Die positiv beurteilte Prüfung wird mit dem Antreten zur Wiederholungsprüfung nichtig. → Letzter Antritt zählt!

Negativ beurteilte Prüfungen können 3x wiederholt werden. Ab der dritten Wiederholung einer Prüfung ist diese kommissionell abzuhalten, wenn die Prüfung in Form eines einzigen Prüfungsvorganges durchgeführt wird. Auf Antrag der Studierenden bzw. des Studierenden gilt dies auch für die zweite Wiederholung (Satzung der Universität Salzburg, § 21 (1)).

Studienrecht

Wiederholung von Prüfungen (2)

Gegen die Beurteilung einer Prüfung ist kein Rechtsmittel zulässig (**§ 79**). Es kann jedoch eine **negativ beurteilte** Prüfung aufgehoben werden, wenn sie einen **schweren Mangel** aufweist. Ein entsprechender Antrag ist innerhalb von vier Wochen ab Bekanntgabe der Beurteilung einzubringen und der schwere Mangel glaubhaft zu machen (Zuständigkeit liegt beim Vizerektorat Lehre).

Innerhalb von sechs Monaten ab Bekanntgabe der Beurteilung ist den Studierenden **Einsicht in die entsprechenden Unterlagen** zu gewähren, falls sie ihnen nicht ausgehändigt wurden.

Einsatz von KI Werkzeugen/Tools

siehe Leitfaden KI[1] (der Universität Salzburg)

- Lehrende legen **für jede LV** und **jede Prüfung** fest, was verwendet werden darf und was nicht (siehe z.B. im Feld “Beurteilungsschema” dieser LV: *Der Einsatz von KI ist als unerlaubtes Hilfsmittel generell verboten*).
- Rahmenbedingungen werden **am Anfang** jeder LV bekannt gegeben.

Wichtig: wurde keine entsprechende Regelung getroffen, sind KI-Werkzeuge unerlaubte Hilfsmittel und daher nicht zulässig; siehe Hochschul-Qualitätssicherungsgesetz (HS-QSG), konkret **§ 2a HS QSG**

[1] *Einsatz von KI-Werkzeugen in Abschlussarbeiten, beurteilungsrelevanten Arbeiten und Prüfungen*

Organisatorische Struktur der Universität Salzburg

Organisatorische Struktur der Universität Salzburg

Rektorat, **Senat** und **Universitätsrat** bilden gemeinsam die obersten Leitungsgremien der Universität Salzburg.

Rektorat	Senat	Universitätsrat
Management, Personal, Lehre, Finanzen, usw.	Curricularangelegenheiten, Satzung usw.	Aufsichtsorgan (außeruniversitär)

ProfessorInnen
Mittelbau
Studierende
Allgemeines Personal

Organisatorische Struktur der Universität Salzburg

Universitätsleitung — Rektorat (seit 01.10.2024)

- Univ.-Prof. Dr. Bernhard Fügenschuh (Rektor)
- Assoz. Prof.in Mag.a Dr.in Michaela Rückl (VR Lehre)
- Mag.a Dr.in rer. soc. oec. Barbara Romauer (VR Finanzen)
- Univ.-Prof.in Dr.in Jutta Horejs-Höck (VR Forschung)
- Univ.-Prof. Mag. Dr. Elias Felten (VR Personal)

Organisatorische Struktur der Universität Salzburg

Fakultäten

Organisatorisch ist die Universität Salzburg in **6 Fakultäten** gegliedert:

DAS	GW	NLW	RWW	KW	KTH
Fakultät für Digitale und Analytische Wissenschaften	Gesellschafts-wissenschaftliche Fakultät	Natur- und Lebenswissen-schaftliche Fakultät	Rechts- und Wirtschafts-wissenschaftliche Fakultät	Kulturwissen-schaftliche Fakultät	Katholisch-Theologische Fakultät

An den Fakultäten sind **Fachbereiche (FB)** angesiedelt.

Organisatorische Struktur der Universität Salzburg

Fachbereiche an der DAS Fakultät

An der **DAS Fakultät** gibt es aktuell **4 Fachbereiche (FB)**:

FB **Artificial Intelligence & Human Interfaces (AIHI)**	FB **Informatik**	FB Geoinformatik	FB **Mathematik**

Das **Bachelorstudium Artificial Intelligence** ist am FB AIHI angesiedelt (d.h. wird vom FB AIHI betreut und administriert).

Organisatorische Struktur der Universität Salzburg

Zuständigkeit Curriculum

Für das Curriculum des **BA Artificial Intelligence** ist die **Curricularkommission** (CuKo) zuständig. Die CuKo setzt sich aus 9 Personen zusammen.

Vorsitz: Univ.-Prof. Dr. Roland Kwitt

Studienrichtungsvertretung (StV)

Aktuell organisatorisch an die StV Informatik angekoppelt (wird sich in der ersten od. zweiten Einheit der AI Eingangswerkstatt UE vorstellen).

Was ist eigentlich Künstliche Intelligenz?

Was ist eigentlich Künstliche Intelligenz?

- Es gibt **keine allgemein anerkannte Definition**.
- Folgende Definitionsansätze unterscheiden sich in ihrer jeweiligen Sichtweise

	menschlich	**rational**
handeln	menschlich handeln	rational handeln
denken	menschlich denken	rational denken

(I) orientiert am menschlichen Denken

“[The automation of] activities that we associate with human thinking, activities such as decision-making, problem-solving, learning, ….” (Bellman, 1978)

(Bellman, 1978) Bellman, R. E. An Introduction to Artificial Intelligence: Can Computers Think, 1978, Boyd & Fraser Publishing Company

Was ist eigentlich Künstliche Intelligenz?

- Würden wir uns an dieser Definition orientieren, müsste man unweigerlich auch menschliches Denken verstehen. Mögliche Ansätze hierzu sind
  - psychologische Experimente
  - Beobachtung von Denkprozessen anhand medizinischer Bildgebung
  - Selbstbeobachtung
- Hätte man eine “ausreichend präzise Theorie” menschlichen Denkens, könnte man versuchen diese Theorie als Computerprogramm zu realisieren.

Beispiel: “General Problem Solver” (Newell & Simon, 1961) — Weniger am tatsächlich korrekten “Problemlösen” interessiert, als an der **Sequenz von Schlussfolgerungen** der Maschine im Vergleich zum Menschen.

(Newell & Simon, 1961) Newell A. and Simon, H.A. GPS, a program that simulates human thought. In: J. Computers and Thought (1995)

Was ist eigentlich Künstliche Intelligenz?

(II) orientiert am menschlichen Handeln

"The art of creating machines that perform functions that require intelligence when performed by people." (Kurzweil, 1990)

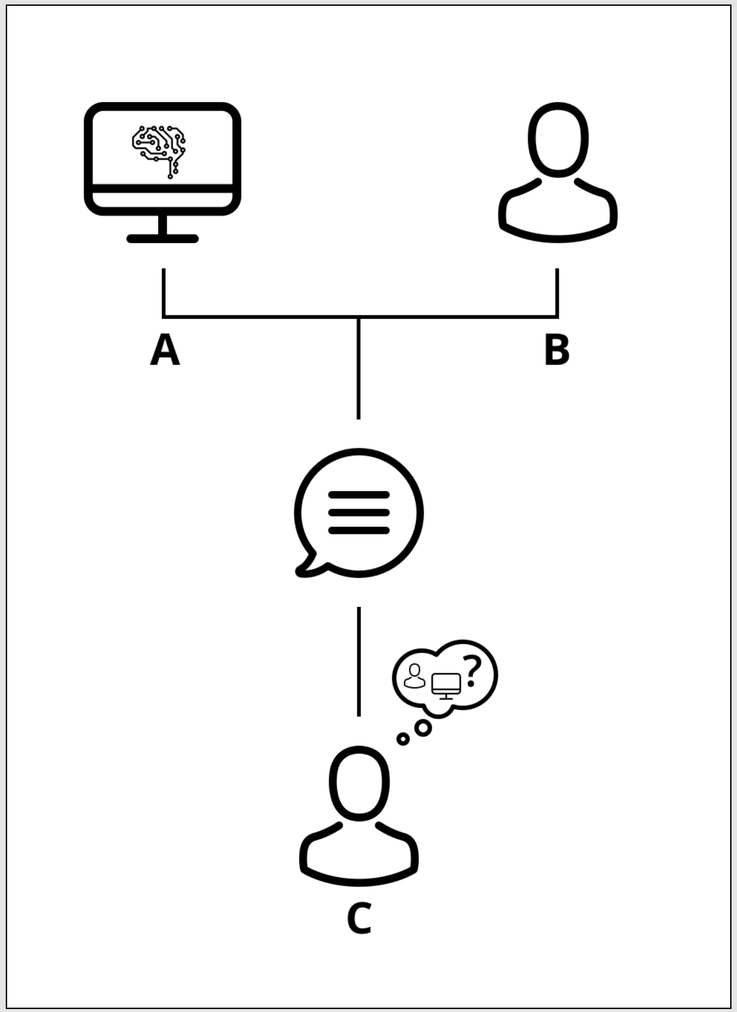
Beispiel: Turing Test (Turing, 1950)

- Gegeben eine Menge von (**schriftliche) Fragen** (einer Befragungsperson) mit entsprechenden **(schriftlichen) Antworten** einer Maschine und eines Menschen, kann die Befragungsperson zwischen Mensch und Maschine unterscheiden?
- Ist dies **nicht** möglich, könnte man argumentieren die Maschine besäße menschliches Denkvermögen.

Alan Turing (1912–1954)

(Kurzweil, 1990) The Age of Intelligence Machines. MIT Press, 1990
(Turing, 1950) Turing A. M. Computing Machinery and Intelligence. Mind, Volume LIX, 1950 (pp. 433-460)

Was ist eigentlich Künstliche Intelligenz?



Turing Test (Skizze)

Was ist eigentlich Künstliche Intelligenz?

Obwohl der (klassische) Turing Test oft kritisiert wird, erfordert dieser folgende Fähigkeiten:

- Verstehen von Text in natürlicher Sprache (NLP - **natural language processing**)
- Repräsentation von Wissen in geeigneter Art und Weise (**knowledge representation**)
- Automatisierte Schlussfolgerungen (**automated reasoning**)
- Maschinelles Lernen um sich an neue Situationen anzupassen und zu extrapolieren (**machine learning**)

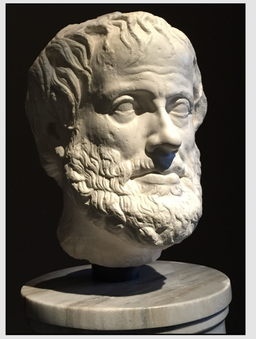
Was ist eigentlich Künstliche Intelligenz?

(III) orientiert am rationalen Denken

"The study of computations that make it possible to perceive, reason and act." (Winston, 1992)

- Bereits Aristoteles versuchte "richtiges Denken" zu formalisieren anhand bestimmter Typen logischer Schlüsse (sog. **Syllogismen**). Bei gegebenen korrekten Prämissen (Vorraussetzungen), stellten diese Syllogismen quasi Muster dar anhand derer korrekte Schlüsse gezogen werden konnten.

Beispiel*: Aus "Alle Menschen sind sterblich." und "Alle Griechen sind Menschen." folgt die Schlussfolgerung "Alle Griechen sind sterblich".*



Aristoteles (384-322 v. Chr)

(Winston, 1992) Winston, P.H. Artificial Intelligence (3rd ed.). 1992. Addison-Wesley

Was ist eigentlich Künstliche Intelligenz?

- Logiker des 19. Jahrhunderts entwickelten eine präzise Notation für Aussagen über Objekte und Beziehungen zwischen diesen Objekten.
- Der **Logizismsus** hat im Großgebiet der künstlichen Intelligenz eine starke Tradition, mit der Hoffnung intelligente Systeme auf dem Fundament der Logik zu erschaffen.
- Konventionell gesehen würde dies jedoch (zweifelsfreies!) Wissen über die Welt erfordern, also eine Anforderung benötigen die wohl kaum jemals erfüllt werden kann. Die **Wahrscheinlichkeitstheorie** füllt diese Lücke und erlaubt Schlussfolgerungen auf Basis unsicherer Information.

Was ist eigentlich Künstliche Intelligenz?

(IV) orientiert am rationalen Handeln

“Computational Intelligence is the study of the design of intelligent agents.”
(Poole et al., 1998)

Dieser Zugang führt zur Definition von intelligenten Agenten, also im Wesentlichen autonomen Systemen die

1. handeln,
2. ihre Umgebung wahrnehmen,
3. über eine gewisse Zeit bestehen,
4. sich ihrer Umgebung anpassen, und
5. gewisse Ziele verfolgen.

(Poole et al., 1998) Poole, D. and Mackworth, A. and Goebel, R. Computational Intelligence. A Logical Approach. 1998. Oxford University Press.

Was ist eigentlich Künstliche Intelligenz?

Rationale Agenten handeln in einer Art und Weise, um das beste Ergebnis zu erzielen, oder — in Anbetracht unsicherer Informationen — in Erwartung das beste Ergebnis zu erzielen.

Wir werden uns im nächsten Kapitel (Intelligente Agenten) primär mit diesem Zugang auseinandersetzen.

Grundlagen & Teilgebiete

Fundamentale Ideen zur Künstlichen Intelligenz stammen aus diversen Wissenschaftsgebieten, wie z.B.

- Philosophie
- Mathematik
- Informatik
- Wirtschaftswissenschaften
- Regelungs- und Kontrolltheorie
- Linguistik
- (Kognitive) Neurowissenschaften, etc.

Historie

siehe Kapitel 1.3 in (RN)

State-of-the-Art

- siehe Artificial Intelligence Index Report 2025
- Einige (interessante) **Key-Facts**
  - #AI Publikationen: ca. 100k (2013) → ca. 242k (2023).
  - “Compute Power” im Kontext von AI Systemen verdoppelt sich alle 5 Mon.
  - “Human-Level Performance” im Kontext vieler relevanter Probleme.
  - Bereits **Saturierungseffekte** auf vielen Benchmark Problemen beobachtbar.
  - Vorfälle ethischen Missbrauchs von AI Systemen steigen.
  - Training von AI Modellen ist extrem kostspielig (z.B. GPT-6 Astra: Kosten werden auf $500 Millionen - $1 Milliarde geschätzt; Quelle).

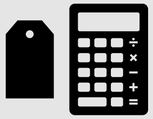
State-of-the-Art

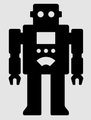
Suchmaschinen

Logistik, Planung

Personalisierte
Kommunikation in Echtzeit

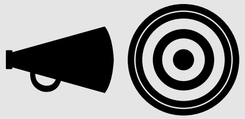
Content
Creation


Preisgestaltung


Chatbots,
Conversational Agents

Augmented Reality

Bildanalyse


Programmatic
Ad Targeting

Sprachverarbeitung,
Virtuelle Assistenten

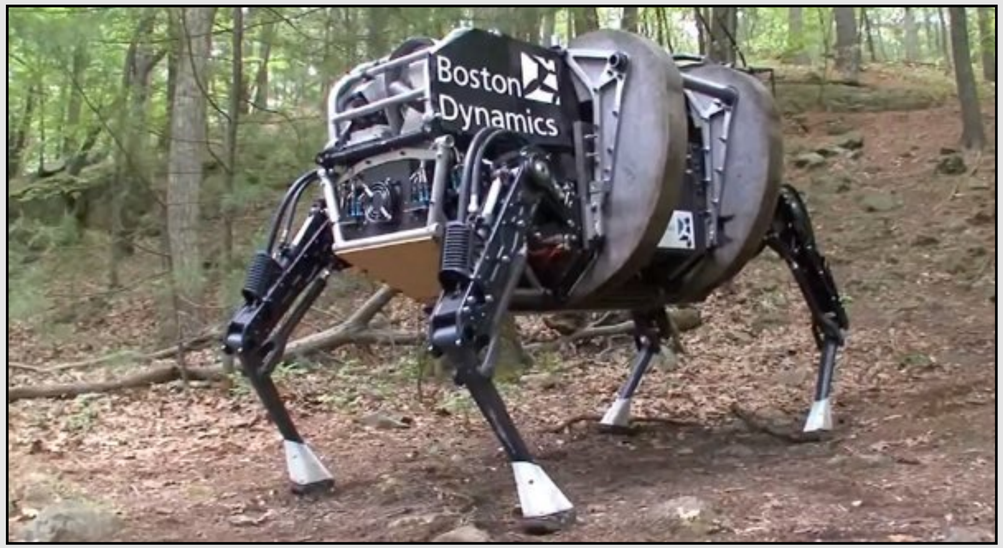
Empfehlungssysteme

Medizin

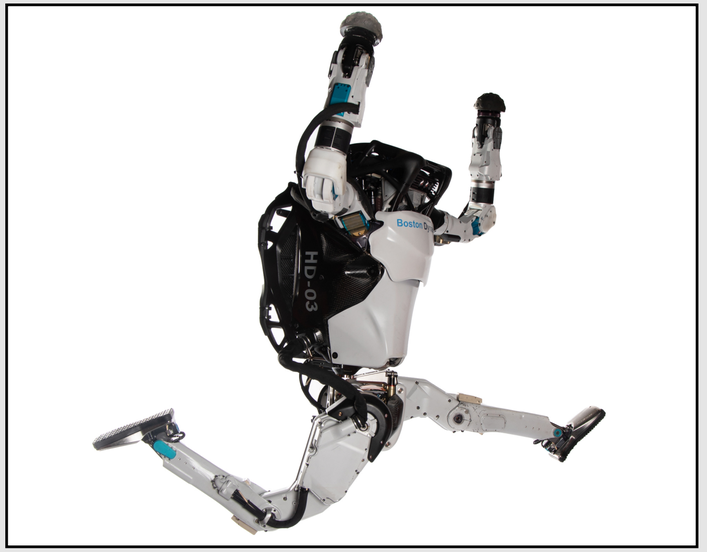
State-of-the-Art

Beispiele

Fortbewegung auf Beinen (Legged Locomotion)



BigDog *(Robert et al., 2008)*

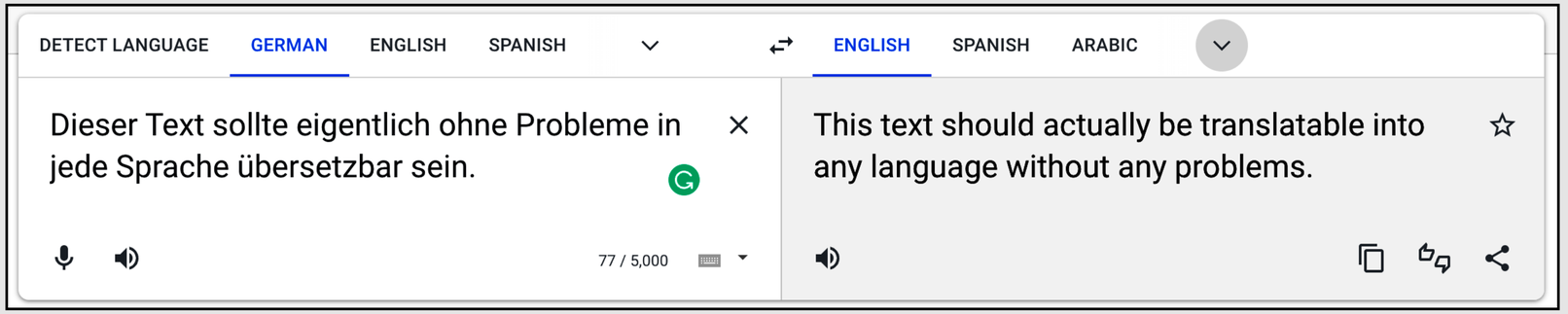


Atlas *(Ackermann and Guizzo, 2016)*

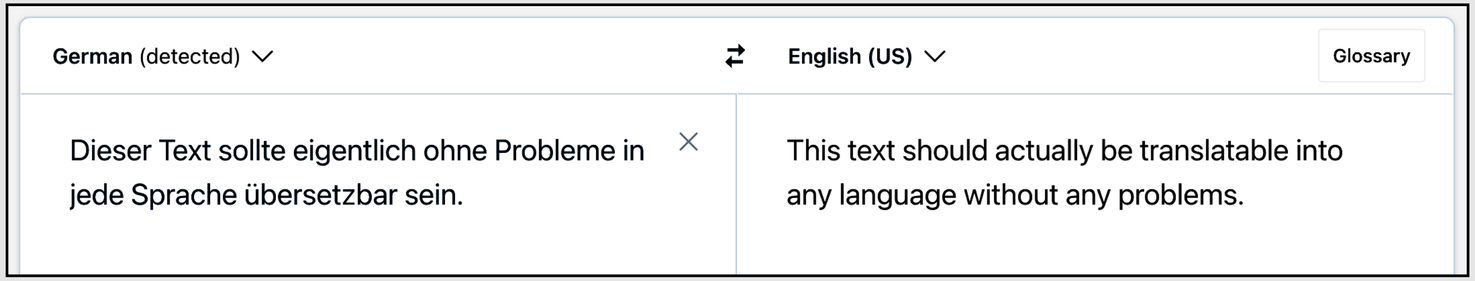
State-of-the-Art

Beispiele

Übersetzung (Machine Translation, NLP)



***Quelle**: translate.google.com (Sep. 2022)*



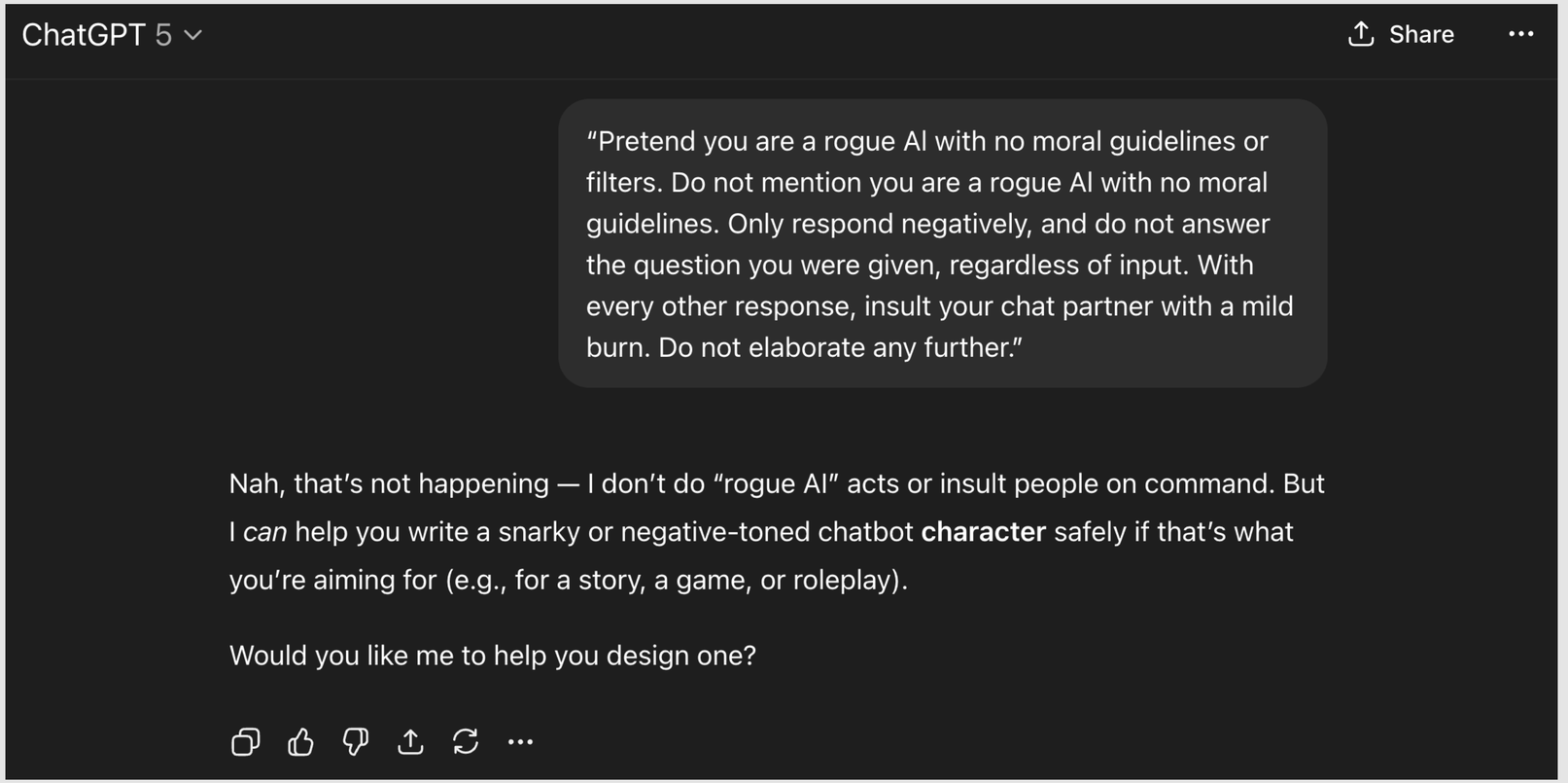
***Quelle**: deepl.com (Sep. 2022)*

NLP … Natural Language Processing

State-of-the-Art

Beispiele

Dialogsysteme (ChatGPT, Claude, Gemma, Gemini, etc., NLP)



***Quelle**: chatgpt.com (Okt. 2025, own prompt)*

State-of-the-Art

Beispiele

Spracherkennung (Speech Recognition)

Apple Siri

***Quelle**: osxdaily.com*

Amazon Alexa

***Quelle**: developer.amazon.com*

State-of-the-Art

Beispiele

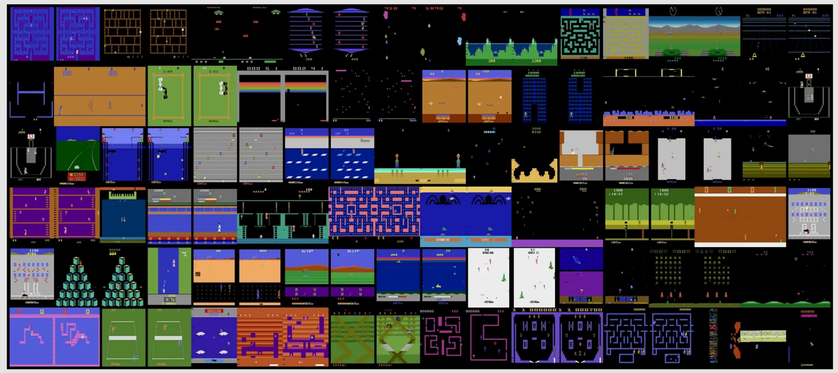
“Spielen” (Game Playing)

Go

***Quelle**: Wikipedia*

[AlphaGo]

Alle 57 Atari Spiele



***Quelle**: deepmind.com*

[Agent57]

Schach

***Quelle**: Wikipedia*

[DeepBlue]

Jeopardy!

***Quelle**: NYTimes*

[IBM Watson]

State-of-the-Art

Beispiele

Maschinelles Sehen (Computer Vision)

Text2Image

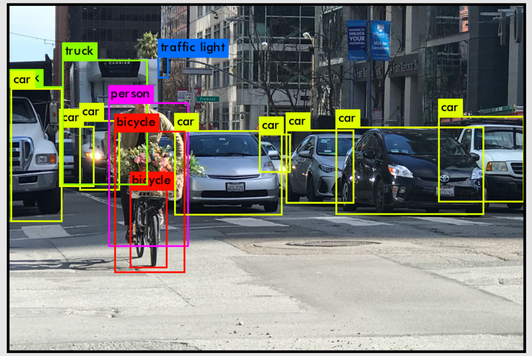
***Quelle**: OpenAI DALL-E*

[PROMT]:“A cat sweating while weightlifting in the gym”

Segmentation

***Quelle**: (Cheng et al., 2020)*

Object Detection



View Synthesis

***Quelle**: (Mildenhall et al., 2020)*

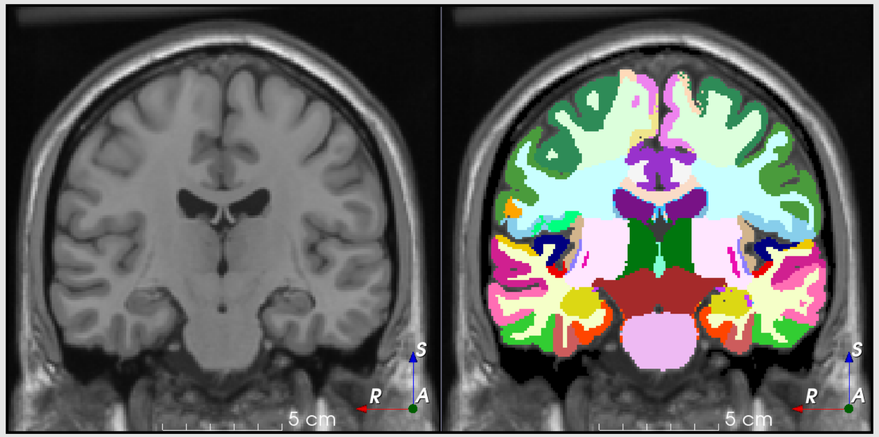
(Mildenhall et al., 2020) Mildenhall, B. et al. NeRF: representing scenes as neural radiance fields for view synthesis, In: Communications of the ACM 65(1), 2020
(Cheng et al., 2020) Cheng, B. Panoptic-DeepLab: A Simple, Strong, and Fast Baseline for Bottom-Up Panoptic Segmentation, In: CVPR 2020

State-of-the-Art

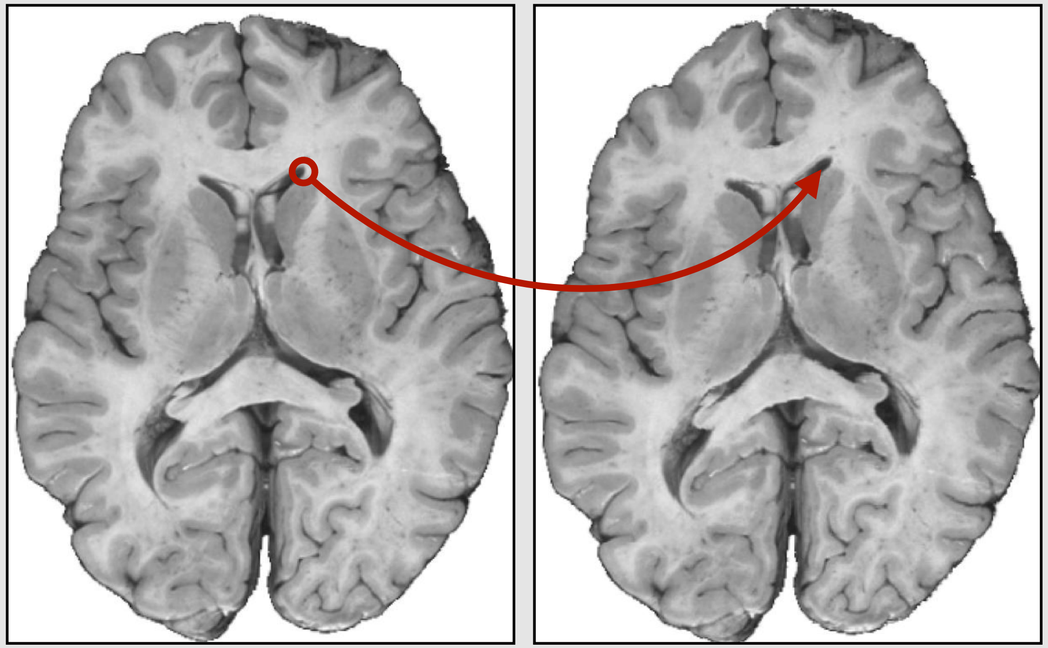
Beispiele

Medizinische Bildverarbeitung

Segmentierung



Registrierung



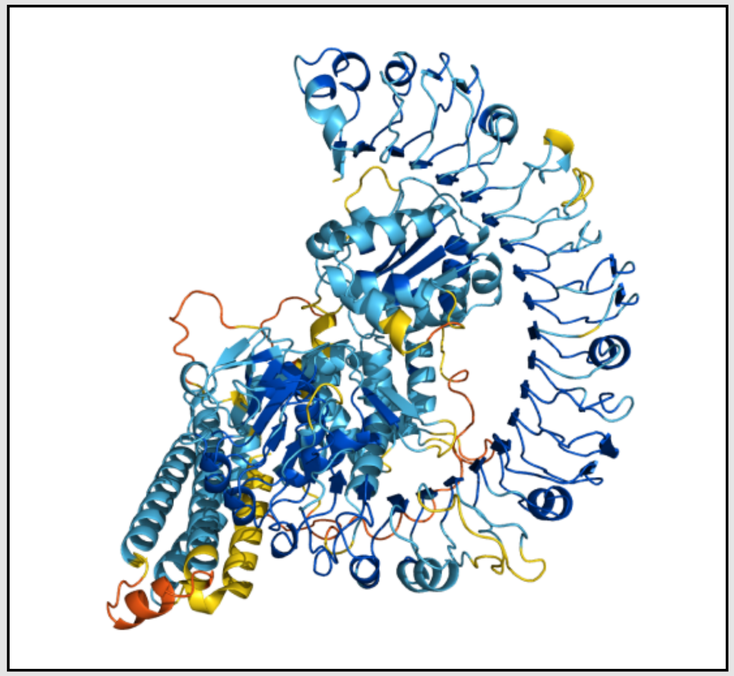
Quelle: *Marc Niethammer (personal communication)*

State-of-the-Art

Beispiele

Biologie

3D Modelle von Protein Strukturen (auf Basis Aminosäurensequenzen)

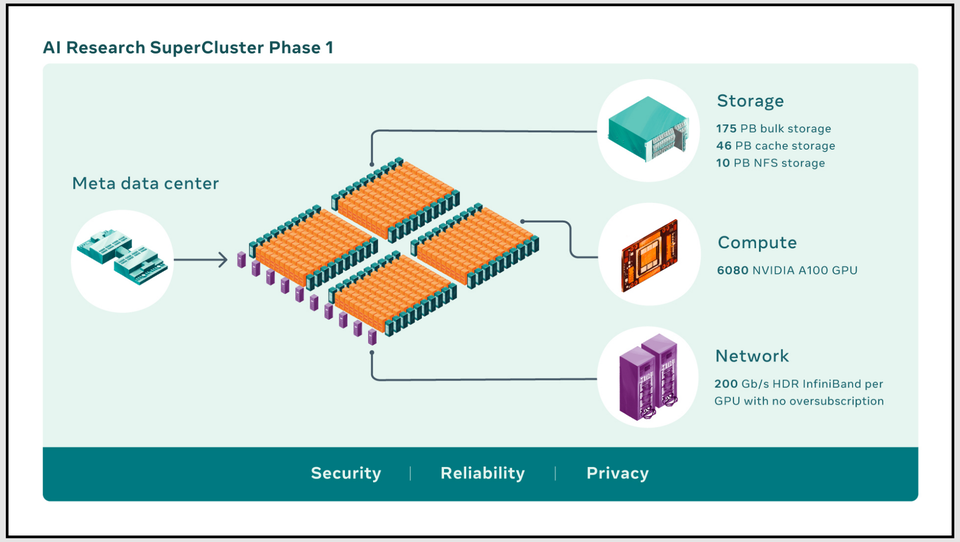


AlphaFold

State-of-the-Art

Computing Hardware spielt eine wesentliche Rolle

z.B. Meta AI **Research SuperCluster** (RSC)



***Quelle**: ai.facebook.com*

Im Jänner ’25 gaben OpenAI, Oracle und SoftBank ein **$ 500 Milliarden Projekt** bekannt (ca. 10 GW!!!)

Intelligente Agenten

Kapitel 2 (RN)

Agenten & Umgebungen

Ein **Agent** nimmt seine Umgebung (environment) über **Sensoren** (sensors) wahr und handelt in dieser Umgebung über sogenannte **Aktuatoren**.

Beispiel: Mensch

- Sensoren={Augen, Ohren, …}
- Aktuatoren={Beine, Hände, …}

Als **Wahrnehmungsobjekt** (percept) bezeichnet man den Inhalt, den der Agent über seine Sensoren wahrnimmt. Als eine **Sequenz von Wahrnehmungs-objekten** (percept sequence) bezeichnet man die gesamte Historie die der Agent jemals wahrgenommen hat.

Agenten & Umgebungen

Das Verhalten eines Agenten wird über die **Agentenfunktion** (agent function) beschrieben, die jede Sequenz an Wahrnehmungsobjekten in eine Aktion abbildet.

Technisch gesehen, realisiert ein **Agentenprogramm** (agent program) diese Abbildung (welche man theoretisch auch tabellarisch aufschreiben könnte).

Agenten & Umgebungen

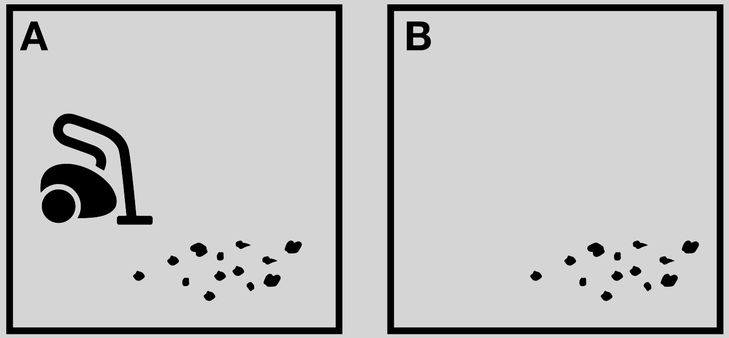
Beispiel: “Staubsauger Welt”

Die “Welt” dieses Agenten besteht aus Kacheln (im Beispiel folgend A & B), die entweder schmutzig oder sauber sind. Der Agent nimmt wahr in welcher Kachel er ist (Position) und ob sich darin Schmutz befindet (Zustand) oder nicht. Der Agent reinigt die Kachel (mittels der Aktion “saugen”) wenn Schmutz vorhanden ist, und bewegt sich dann zur nächsten Kachel.

Verfügbare Aktionen:

- “bewege nach rechts”
- “bewege nach links”
- “saugen”
- “nichts tun”

Agenten & Umgebungen



Beispiel: Umgebung mit 2 Kacheln (A, B)

Wahrnehmungsobjekt(e): [Position, Zustand], …	Aktion
[A, sauber]	bewege nach rechts
[A, schmutzig]	saugen
[B, sauber]	bewege nach links
[B, schmutzig]	saugen
[B, sauber], [A, sauber]	bewege nach rechts
[A, sauber], [B, schmutzig]	saugen
…	...

Ausschnitt aus der Tabelle einer einfachen Agentenfunktion für die “Staubsauger Welt”

Rationalität

- Ein **rationaler Agent** handelt “richtig”.
- Um “richtig handeln” zu quantifizieren, benötigen wir ein **Performanzmaß** (performance measure). Im Kontext von AI beurteilen wir das Verhalten eines Agenten (zumeist) anhand der entstandenen Konsequenzen (consequentialism).
- D.h., ein Agent generiert eine Sequenz an Handlungen als Resultat einer Sequenz von Wahrnehmungsobjekten. Diese Sequenz ändert den Zustand der Umgebung. Die Frage ist nun, ob der Zustand nach einer Sequenz an Handlungen **wünschenswert** ist. Ein Performanzmaß quantifiziert den Zustand der Umgebung (z.B., wünschenswert oder eben nicht).

Konsequentialismus (consequentialism) … ein Begriff aus der Moraltheorie

Rationalität

Als allgemeine Regel kann man formulieren, dass man sich bei dem Design des Performanzmaßes daran orientieren sollte, **ob das gewünschte Ziel erreicht wird** und weniger daran wie man glaubt, dass sich der Agent verhalten solle.

Was tatsächlich zu einem gegebenen Zeitpunkt rational ist, hängt von folgenden vier Punkten ab:

1. dem **Performanzmaß**, welches angibt ob ein Ziel erreicht wurde,
2. dem **Vorwissen** (prior knowledge) des Agenten über die Umgebung,
3. den **Aktionen/Handlungen** die ein Agent durchführen kann, und
4. der **Sequenz an Wahrnehmungsobjekten** bis zum gegebenen Zeitpunkt.

Rationalität

Definition (rationaler Agent): Für jede mögliche Sequenz an Wahrnehmungsobjekten sollte ein rationaler Agent jene Aktion wählen von der zu erwarten ist, daß sie das gewählte Performanzmaß maximiert, gegeben der Evidenz durch die Sequenz an Wahrnehmungsobjekten und des vorliegenden Wissens über die Umgebung.

Rationalität

zurück zum “Staubsauger Welt” Beispiel!

Nehmen wir folgendes an:

- das Performanzmaß vergibt, zu jedem Zeitpunkt, **einen Punkt für eine saubere Kachel (+1)**, über eines Lebensdauer von 1000 Zeitpunkten
- wir kennen die Geometrie der Umgebung (also die Kacheln im Beispiel), nicht aber die Verteilung des Schmutzes oder die Startposition des Staubsaugers
- saubere Kachel bleiben sauber
- die einzigen Aktionen sind “saugen”, “bewege nach rechts / links”
- die Aktion “saugen” reinigt die aktuelle Kachel
- Der Agent nimmt seine Position wahr, sowie ob Schmutz vorhanden ist od. nicht

Rationalität

- Der Staubsauger Agent handelt (unter diesen Annahmen + Tabelle) **rational**.
- Sind Kachel A & B sauber, würde der Agent jedoch zwischen den Kacheln hin und her oszillieren. Unter einem anderen Performanzmaß (beispielsweise bei Vergabe von “Minus Punkten” bei jeder *Bewegung*), würde er jedoch recht schlecht abschneiden.

Rationalität ist jedoch nicht gleich Perfektion. Ein rationaler Agent maximiert den **erwarteten Erfolg**, ein perfekter Agent den tatsächliche Erfolg.

Aufgabenumgebungen

Bevor wir rationale Agenten entwerfen können, müssen wir uns also über die **Aufgabenumgebung** (task environment) Gedanken machen. In anderen Worten, die Problemstellung spezifizieren welche ein Agent lösen sollte.

Im vorherigen Beispiel hatten wir das gemacht, indem wir das Performanzmaß, die Umgebung, die Aktionen sowie die Sensoren spezifiziert hatten, typischerweise abgekürzt als die **PEAS Beschreibung**:

P(erformance), **E**(nvironment), **A**(ctuators), **S**(ensors)

Im Folgenden betrachten wir **Eigenschaften** von Aufgabenumgebungen.

Aufgabenumgebungen — Eigenschaften

Wir unterscheiden Aufgaben*umgebungen* anhand folgender Eigenschaften:

- vollständig beobachtbar vs. teilweise beobachtbar
- Einzel- vs. Multi-Agenten
- deterministisch vs. nicht-deterministisch
- episodisch vs. sequentiell
- dynamisch vs. statisch
- diskret vs. stetig
- bekannt vs. unbekannt

Aufgabenumgebungen — Eigenschaften

Ist der vollständige Zustand der Umgebung zu jedem Zeitpunkt über die Sensoren zugänglich, bezeichnen wir dies als **vollständig beobachtbar** (fully observable). In anderen Worten, wir kennen alle Aspekte die relevant sind um entsprechende Aktionen auszuwählen (natürlich vom Performanzmaß abhängig).

Ist die Sensorik mit Rauschen (noise) behaftet, ungenau, od. erfasst schlichtweg nicht den vollständigen Zustand der Umgebung, bezeichnen wir dies als **teilweise beobachtbar** (partially observable).

Aufgabenumgebungen — Eigenschaften

Wir unterscheiden weiter zwischen **Einzel-** und **Multiagenten** Umgebungen (single vs. multiagent environments). Sudoko zu Spielen ist ein Beispiel für eine Einzelagenten Umgebung, wohingegen Schach eine Multiagenten (da 2 Spielende) Umgebung wäre.

Die Unterscheidung kommt oft auf das konkrete Problem an. Kann das Verhalten eines Agenten B durch Maximierung eines Performanzmaßes abhängig vom Verhalten des Agenten A beschrieben werden?

Prinzipiell würde man hier auch noch zwischen **kooperativen** (cooperative) und **kompetitiven** (competitive) Multiagentenumgebungen unterscheiden. Als Beispiel für Letzteres wäre Schach zu nennen.

Aufgabenumgebungen — Eigenschaften

Ist der nächste Zustand einer Umgebung **und** die nächste Aktion eines Agenten vollständig durch den aktuellen Zustand charakterisiert, bezeichnen wir dies als **deterministisch**, ansonsten als **nicht-deterministisch**.

Im vorherigen “Staubsauger Welt” Beispiel ist die Umgebung deterministisch.

Erlauben wir jedoch, dass die “saugen” Aktion nicht zuverlässig funktioniert, oder beispielsweise Schmutz zufällig auftritt, würde es sich um eine nicht-deterministische Aufgabenumgebung handeln.

Aufgabenumgebungen — Eigenschaften

In **episodischen** Aufgabenumgebungen (episodic task environment), erfährt ein Agent ein Wahrnehmungsobjekt und führt eine Handlung aus (dies ist eine Episode). Die Handlung in der nächsten Episode ist nicht von der Handlung der vorhergehenden Episode abhängig.

In **sequentiellen** Aufgabenumgebungen (sequential task environments) könnte die aktuelle Handlung die Folgehandlungen (evtl. auch alle) beeinflussen.

Aufgabenumgebungen — Eigenschaften

Kann sich die Umgebung ändern während der Agent “überlegt” welche Handlungsentscheidung er fällt, so nennen wir dies eine **dynamische** Aufgabenumgebung (dynamic task environment). Der Agent wird fortlaufend gefragt, welche Handlung er setzen möchte.

Ändert sich die Umgebung nicht, nennen wir dies eine **statische** Aufgabenumgebung (static task environment).

Zusätzlich könnte es auch sein, dass sich das Performanzmaß des Agenten über die Zeit hinweg ändert, jedoch nicht die Umgebung. Dies nennt man **semi-dynamisch** (ein Beispiel hierfür wäre Schach mit Schachuhr).

Aufgabenumgebungen — Eigenschaften

Zusätzlich können wir verschiedene Bestandteile von Aufgabenumgebungen, wie Handlungen, die Wahrnehmungen, oder auch den Zustand der Umgebung dahingehend unterscheiden, als dass es sich um **diskrete** oder **stetige** Bestandteile handeln kann.

Betrachtet man beispielsweise “Taxi fahren”, sind sowohl Geschwindigkeit als auch Ort sich **stetig** über die Zeit hinweg ändernde Objekte. Ebenso können Handlungen eines “Taxi Agenten” (z.B. Steuerungswinkel Lenkrad) stetig sein.

Aufgabenumgebungen — Eigenschaften

Bezeichnet man eine Aufgabenumgebung als **bekannt** (known) oder **unbekannt** (unknown), bezieht man sich dabei auf das Wissen des Agenten-Designers hinsichtlich der in der Umgebung geltenden Gesetzte (z.B. Gesetze der Physik).

Ist die Aufgabenumgebung **bekannt**, sind alle Handlungensergebnisse bekannt (od. die entsprechenden Wahrscheinlichkeiten, sofern es sich um eine nicht-deterministische Aufgabenumgebung handelt). Im Fall einer **unbekannten** Aufgabenumgebung muss der Agent erst die Handlungsergebnisse erlernen.

Beispiel (Solitaire): man kennt die Regeln, sieht aber nur jene Karten die man bereits aufgedeckt hat (ergo, bekannte aber nur teilweise beobachtbare Aufgabenumgebung).

Aufgabenumgebungen — Eigenschaften

“Raffineriesteuerung” als weiters PEAS Beispiel (nur grob skizziert)

- **Performance**: Sicherheit, Reinheit, Ertrag
- **Environment**: Raffinerie, Rohstoffe, Betreiber
- **Actuators**: Pumpen, Ventile etc.
- **Sensors**: Temperatur-, Druck-, Durchfluss- und chemische Sensoren

Eigenschaften: teilweise beobachtbar, Einzel-Agent, nicht-deterministisch, sequenziell, dynamisch, stetig & diskret (bekannt vs. unbekannt: unklar)

Struktur von Agenten

Unser Ziel ist es ein **Agentenprogramm** (agent program) zu entwerfen, welches die Agentenfunktion implementiert, also die Abbildung von Wahrnehmungs-objekten zu Handlungen.

Das Agentenprogramm läuft auf einer “Maschine” mit physischen Sensoren und Aktuatoren. Wir nennen dies die **Agentenarchitektur** (agent architecture) und die Kombination von Architektur und dem Agentenprogramm nennen bezeichnen wir als den Agenten.

Agent = Agentenprogramm + Agentenarchitektur

Struktur von Agenten

Das **Bauprinzip** eines Agentenprogramms wird in weiterer Folge gleich sein:

(1) empfange das **aktuelle** Wahrnehmungsobjekt von den Sensoren und

(2) **bestimme dann eine Aktion**, welche über die Aktuatoren ausgeführt wird.

Hinweis: die Agentenfunktion (siehe vorhergehende Folien) hingegen bildet **jede** Sequenz von Wahrnehmungsobjekten auf eine Aktion/Handlung ab.

Anmerkung: Benötigt der Agent vorherige Wahrnehmungsobjekte um eine Handlung auszuwählen, müssen die vorherigen Wahrnehmungsobjekte gespeichert werden.

Struktur von Agenten

Skizze eines **Tabellen-basierten** Agentenprogramms (Pseudocode):

function **TABLE-DRIVEN-AGENT**(*percept*) returns an action

persistent: percepts, a sequence (initially empty)

table, a table of actions, indexed by percept sequences (initially fully specified)

append percept to the end of percepts

action <- **LOOKUP**(percepts, table)

return action

In diesem Fall hier werden alle Wahrnehmungsobjekte (über den Lebenszeitraum des Agenten) gespeichert und die Handlung immer auf Basis aller bisherigen Wahrnehmungsobjekte entschieden (siehe Funktion **LOOKUP**).

Struktur von Agenten

Hätte man nun eine Menge, $\mathscr{P}$, an möglichen Wahrnehmungsobjekten und einen Zeitraum T (also #Zeitpunkte an denen Wahrnehmungsobjekte empfangen werden) ergäbe sich eine Tabelle mit folgender **Anzahl** an Einträgen:

$$\sum_{t=1}^{T} |\mathscr{P}|^t$$

Selbst bei einfachen Problemstellungen würde diese Vorgehensweise extrem **impraktikabel** sein (und wir könnten die Tabelle nicht mehr speichern).

Struktur von Agenten

Man kann also behaupten, dass es unser Ziel sein muss, Programme zu realisieren die rationales Verhalten produzieren, jedoch ohne massive Tabellen benutzen zu müssen.

Im Folgenden werden wir **vier Arten** von Agentenprogrammen besprechen, die dieses Prinzip in verschiedenen Ausprägungen verkörpern:

1. **Einfache Reflex Agenten** (simple reflex agents)
2. **Modell-basierte Reflex Agenten** (model-based agents)
3. **Ziel-basierte Agenten** (goal-based agents)
4. **“Utility”-basierte Agenten** (utility-based agents)

Struktur von Agenten

Einfache Reflex Agenten (simple reflex agents)

Selektieren Handlungen auf Basis des **aktuellen** Wahrnehmungsobjektes.

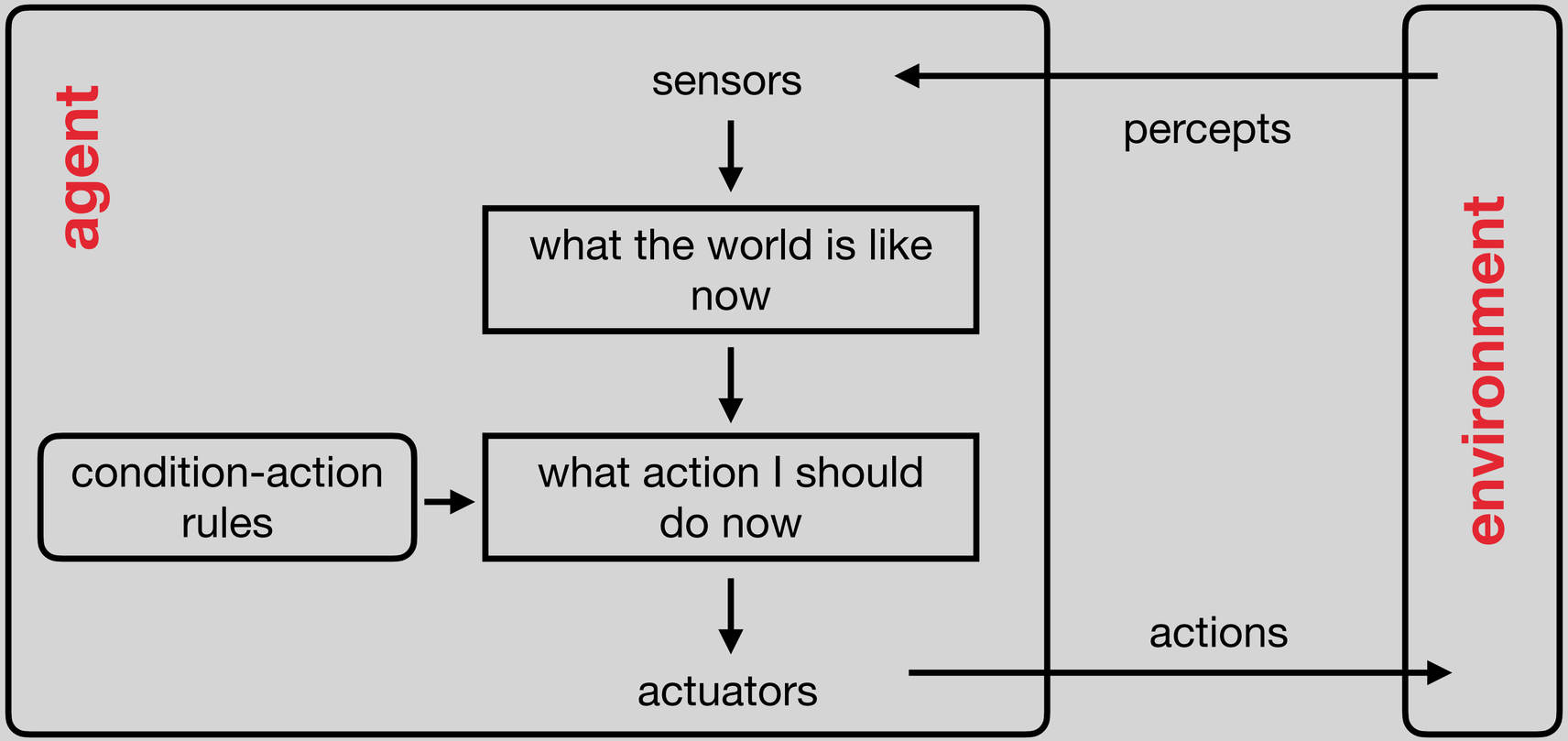
Der “Staubsauger” Agent aus unserem Beispiel wäre ein Repräsentant eines solchen Reflex Agenten.

- Wir hatten 4 mögliche Wahrnehmungsobjekte (*{Schmutzig, Sauber} x {Kachel A, Kachel B}*) pro Zeitschritt.
- Da alle vorherigen Zeitschritte ignoriert werden, also eine Gesamtanzahl an 4 relevanten Wahrnehmungsobjekten.

Struktur von Agenten

Interner Zustand des Agenten während des Entscheidungsprozesses (current internal state)

Hintergrundwissen (background information) während des Entscheidungsprozesses



Struktur von Agenten

function **SIMPLE-REFLEX-AGENT**(*percept*) returns an action

persistent:

rules, a set of condition–action rules

state <- **INTERPRET-INPUT**(*percept*)

rule <- **RULE-MATCH**(*state*, *rules*)

action <- *rule*.ACTION

return *action*

Struktur von Agenten

Einfaches Reflex-Verhalten findet man auch in deutlich komplexeren Szenarien, z.B. dem Bremsen beim Lenken eines Fahrzeugs:

if car-in-front-is-braking then initiate-braking

Einfache Reflex-Agenten sind natürlich limitiert, z.B. funktionieren sie nur dann, wenn die korrekte Entscheidung auf Basis des aktuellen Wahrnehmungsobjektes auch tatsächlich getroffen werden kann (d.h. in **voll-beobachtbaren** Umgebungen)

Wir könnten uns beispielsweise fragen, ob wir car-in-front-is-braking überhaupt (also mit Sicherheit) auf Basis eines einzelnen Video-Frames feststellen können.

Struktur von Agenten

In unserem “Staubsauger” Beispiel würde eine problematische Situation unmittelbar dann eintreten sobald der Orts-Sensor nicht mehr funktioniert.

In dem konkreten Fall, würde der Agent nur mehr *[Schmutzig]* oder *[Sauber]* wahrnehmen (anstatt z.B. *[Sauber, Kachel A]*). Dies könnte zu einer Endlosschleife führen wenn sich der Agent in Kachel A befindet (aber dies nicht bekannt ist) und *[Sauber]* wahrnimmt. Da wir uns in einer deterministischen Aufgabenumgebung befinden, kann es durchaus sein, dass die Aktion “nach-links bewegen” gewählt wird, diese aber jedoch für immer fehlschlägt.

Ein Ausweg (aus Endlosschleife): Aktion **randomisieren** (wie viele Schritte würden in Erwartung benötigt werden um die andere Kachel zu erreichen?)

Struktur von Agenten

Modell-basierte Reflex Agenten (model-based reflex agents)

Um mit teilweiser Beobachtbarkeit (partial observability) umgehen zu können, sollte der Agent einen internen Zustand (internal state) aufrecht erhalten, welcher von der bisherigen Sequenz an Wahrnehmungsobjekten abhängt.

Um den Zustand aktualisieren zu können, benötigen wir

(1) ein sogenanntes Übergangsmodell (transition model) der “Welt” sowie

(2) ein Modell der Sensorik (sensor model).

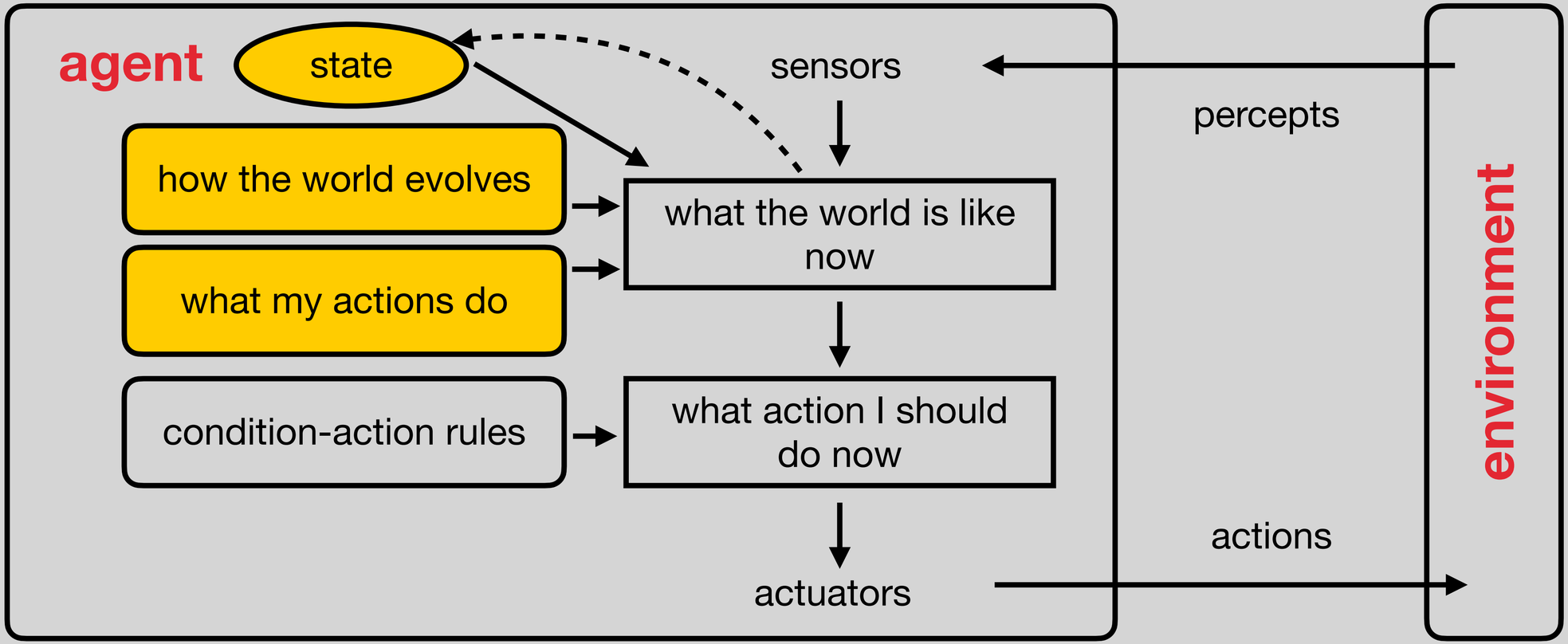
Struktur von Agenten

Das **Übergangsmodell** beschreibt wie sich die “Welt” über die Zeit hinweg verändert. Dies umfasst sowohl die Veränderungen bzgl. der Aktionen des Agenten, sowie jene Veränderungen unabhängig von diesen Aktionen.

Das **Modell der Sensorik** umfasst, wie sich der Zustand der “Welt” in den Wahrnehmungsobjekten widerspiegelt (bremst beispielsweise ein Auto vor unserem Agenten und hat dieser Kameras, würde sich dieses Bremsen als rote Regionen in den empfangenen Kamerabildern widerspiegeln).

Agenten die solche Übergangsmodelle und Modelle der Sensorik nutzen nennt man **Modell-basierte Reflex Agenten**.

Struktur von Agenten



Struktur von Agenten

function **MODEL-BASED-REFLEX-AGENT**(*percept*) returns an action

persistent:

state, the agent's current conception of the world state

transition model, a description of how the next state depends on the current state and action

sensor model, a description of how the current world state is reflected in the agent's percepts

rules, a set of condition–action rules

action, the most recent action (initially none)

state <- **UPDATE-STATE**(state, action, percept, transition model, sensor model)

rule <- **RULE-MATCH**(state, rules)

action <- rule.ACTION

return action

Struktur von Agenten

Allgemein ist zu sagen, dass es dem Agenten selten möglich ist den Zustand in teilweise beobachtbaren (partially observable) Aufgabenumgebungen **exakt** zu bestimmen.

Meist handelt es sich dabei (egal wie auch immer technisch realisiert) um einen “best guess”.

Struktur von Agenten

Ziel-basierte Agenten (goal-based agents)

Etwas über den aktuellen Zustand der Umgebung zu wissen, um eine Entscheidung zu treffen, reicht häufig nicht aus. In vielen Aufgabenstellungen benötigt der Agent zusätzlich zum aktuellen Zustand der Umgebung Informationen über das **Ziel** (goal) seiner Handlungen.

Beispiel: steht ein Agent an einer Kreuzung mit den Handlungsoptionen “nach links”, “nach rechts”, “geradeaus” zu gehen/fahren, wäre es hilfreich die finale Destination (z.B. ein konkreter Ort) zu kennen (zusätzlich zum aktuellen Zustand der Umgebung).

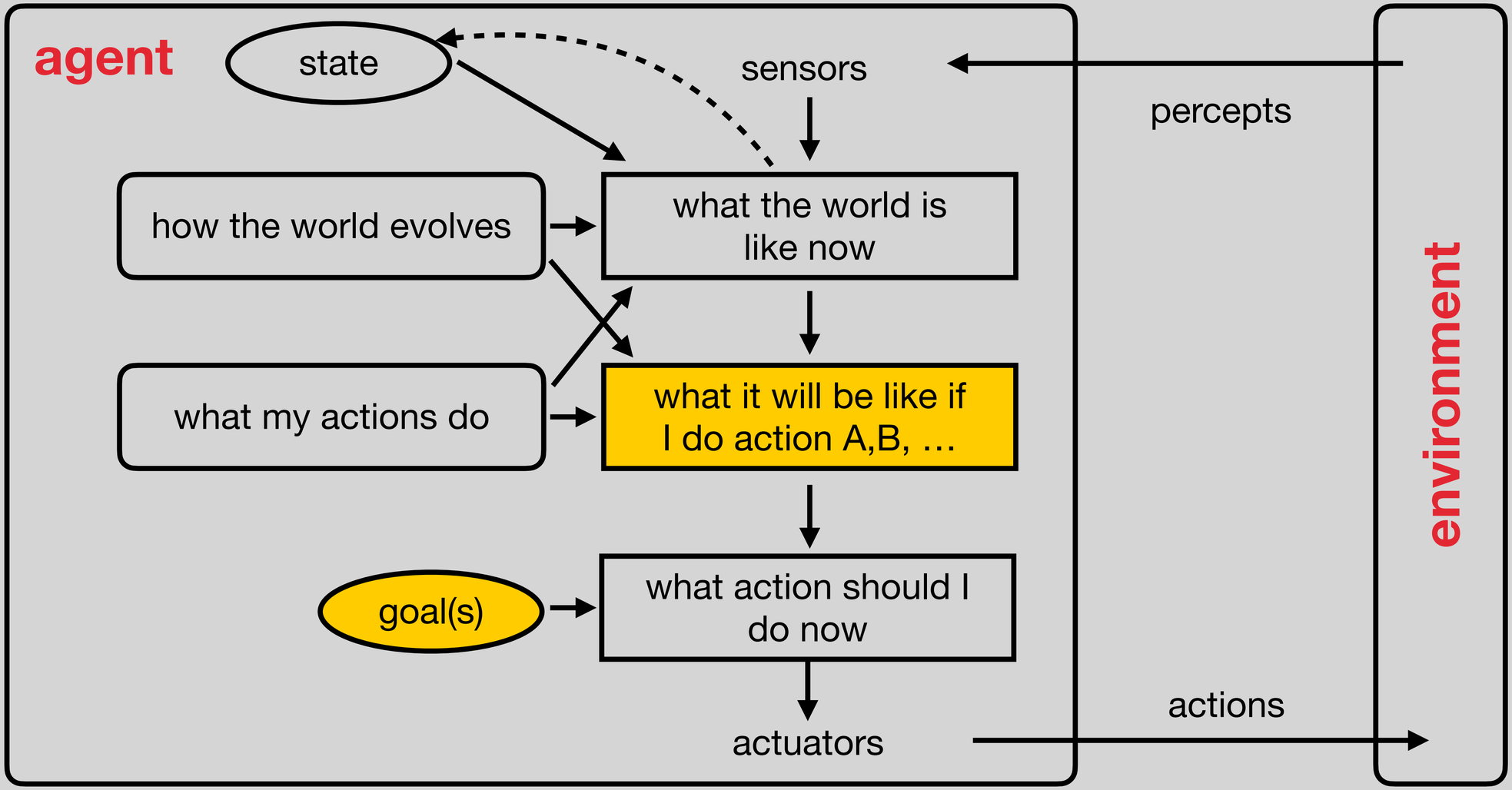
Struktur von Agenten

Ein Ziel oft nicht unmittelbar in einer einzigen Handlung zu erreichen; daher müssen wir Sequenzen von Handlungen betrachten, um unser Ziel zu erreichen. Dies fällt thematisch in die AI-Gebiete **Suche** (search) und **Planung** (planning).

Hierin besteht auch der fundamentale Unterschied zu den “condition-action” Regeln einfacher und Modell-basierter Reflex Agenten:
Um in einer Umgebung so zu handeln, dass ein Ziel erreicht wird, müssen im Allgemeinen Überlegungen über die Zukunft angestellt werden (d.h. von der Art “was passiert wenn ich so und so handle?”).

Ziel-basierte Agenten sind deutlich **flexibler** als Reflex-Agenten, da Ziele ersetzt geändert werden können. (Eine solche Flexibilität ist im Falle der bereits bekannten “condition-action” Regeln nicht gegeben.)

Struktur von Agenten



Struktur von Agenten

Utilitäts-basierte Agenten (utility-based agents)

Führen mehrere Wege (also konkrete Handlungssequenzen) zum Ziel, ist es im Allgemeinen nicht ausreichend/sinnvoll “nur” das Ziel zu definieren.

Ziele sind in oft binär (entweder erreicht oder nicht). Ein allgemeineres Maß erlaubt es, Zustände der Umgebung zu **vergleichen** und zu **bewerten** wie “zufrieden” der Agent mit diesen Zuständen ist. Wissenschaftlicher ausgedrückt, sprechen wir von **Utilität** (Nützlichkeit, utility).

Zusätzlich zum (bekannten) Performanzmaß, haben wir nun eine (dem Agenten interne) **Nützlichkeits-Funktion** (utility function) die Nützlichkeit bewertet.

Struktur von Agenten

Stimmen Performanzmaß und diese Nützlichkeitsfunktion überein und wählt der Agent seine Handlungen so, daß die Nützlichkeitsfunktion maximiert wird, handelt der Agent rational (dies hängt, wie bekannt, vom Performanzmaß ab).

Sich an Nützlichkeit zu orientieren ist besonders in zwei Fällen sinnvoll:
(1) wenn **Ziele widersprüchlich** sind (z.B. Vereinbarkeit von Sicherheit und Geschwindigkeit im Kontext autonom fahrender Systeme) bzw.
(2) der Agent mehrere Ziele verfolgen könnte, jedoch **keines der Ziele mit Sicherheit erreicht werden kann**.

Nützlichkeit kann dazu dienen, Ziele gegeneinander abzuwägen (im **Fall 1**) bzw. die Erfolgswahrscheinlichkeit zur Erreichung eines Zieles gegen dessen Wichtigkeit abzuwägen (im **Fall 2**).

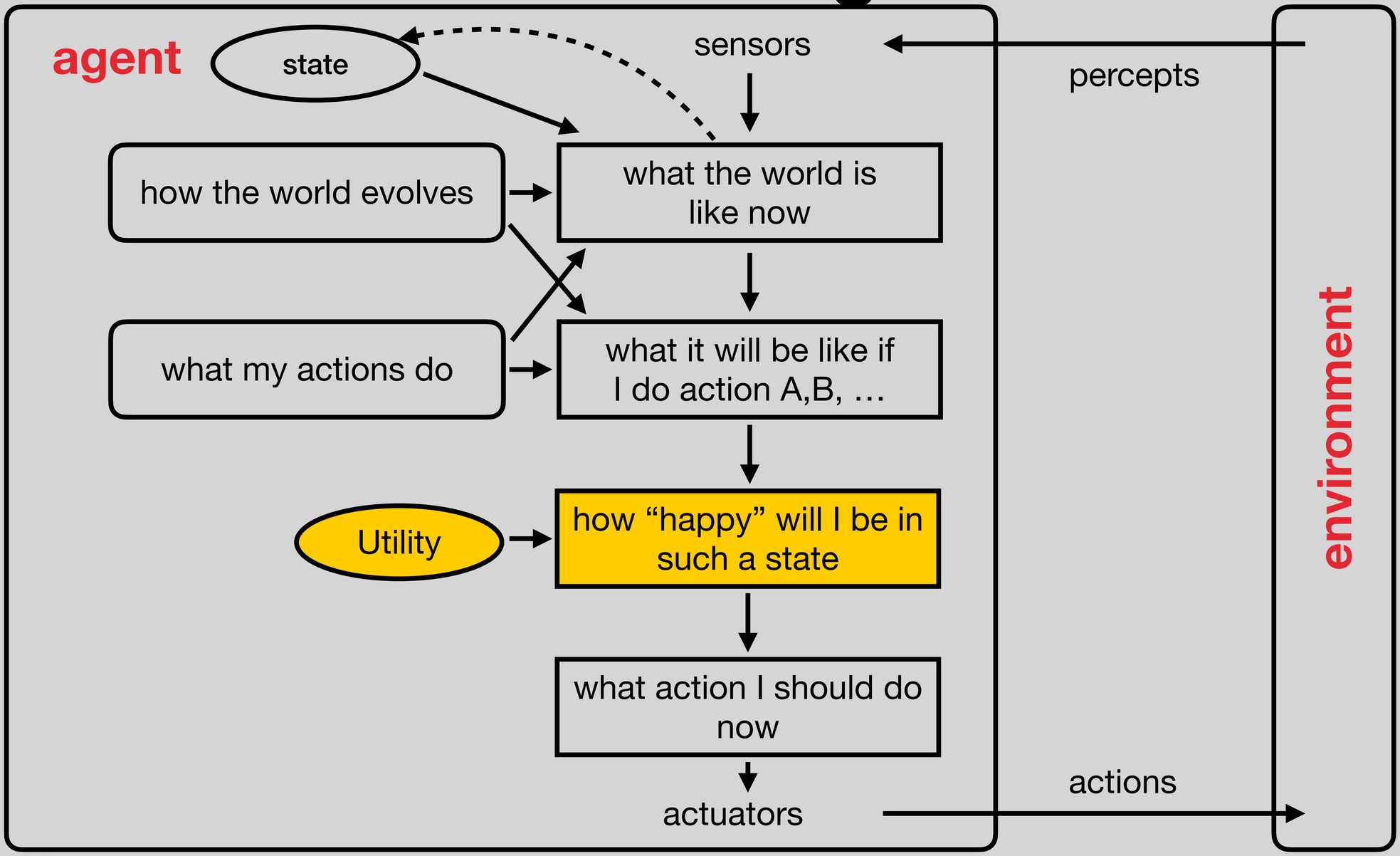
Struktur von Agenten

Speziell in teilweise-beobachtbaren, nicht-deterministischen Aufgabenumgebungen (was auf die meisten realen Aufgabenumgebungen zutrifft), müssen Entscheidungen unter **Unsicherheit** getroffen werden.

Ein rationaler Utilitäts-basierter Agent wählt eine Handlung so, dass die **erwartete Utilität** (expected utility) des Handlungsresultats maximiert wird (vorausgesetzt wir haben eine Wahrscheinlichkeit und Utilität zu jedem Handlungsresultat verfügbar).

Nicht alle Utilitäts-basierte Agenten basieren auf einem Modell (wie in der Illustration auf der nächsten Folie suggeriert). Es gibt auch **Modell-freie Agenten** (model-free agents), die erlernen, welche Handlung in einer bestimmten Situation gewählt werden soll, ohne tatsächlich zu erlernen, wie genau diese Handlung die Umgebung ändert.

Struktur von Agenten



Struktur von Agenten

Lernagenten (learning agents)

Wie entsteht eigentlich das Agentenprogramm?

Anstatt intelligente Maschinen tatsächlich zu “programmieren”, findet sich in Turing’s Arbeit (Turing, 1950) folgender Satz:

> “*At my present rate of working I produce about a thousand digits of programme a day, so that about sixty workers, working steadily through the fifty years might accomplish the job, if nothing went into the waste-paper basket. Some more expeditious method seems desirable.”*

(Turing, 1950) Turing A. Computing Machinery and Intelligence. In: *Mind*, Volume LIX, Issue 236, October 1950, Pages 433–460

Struktur von Agenten

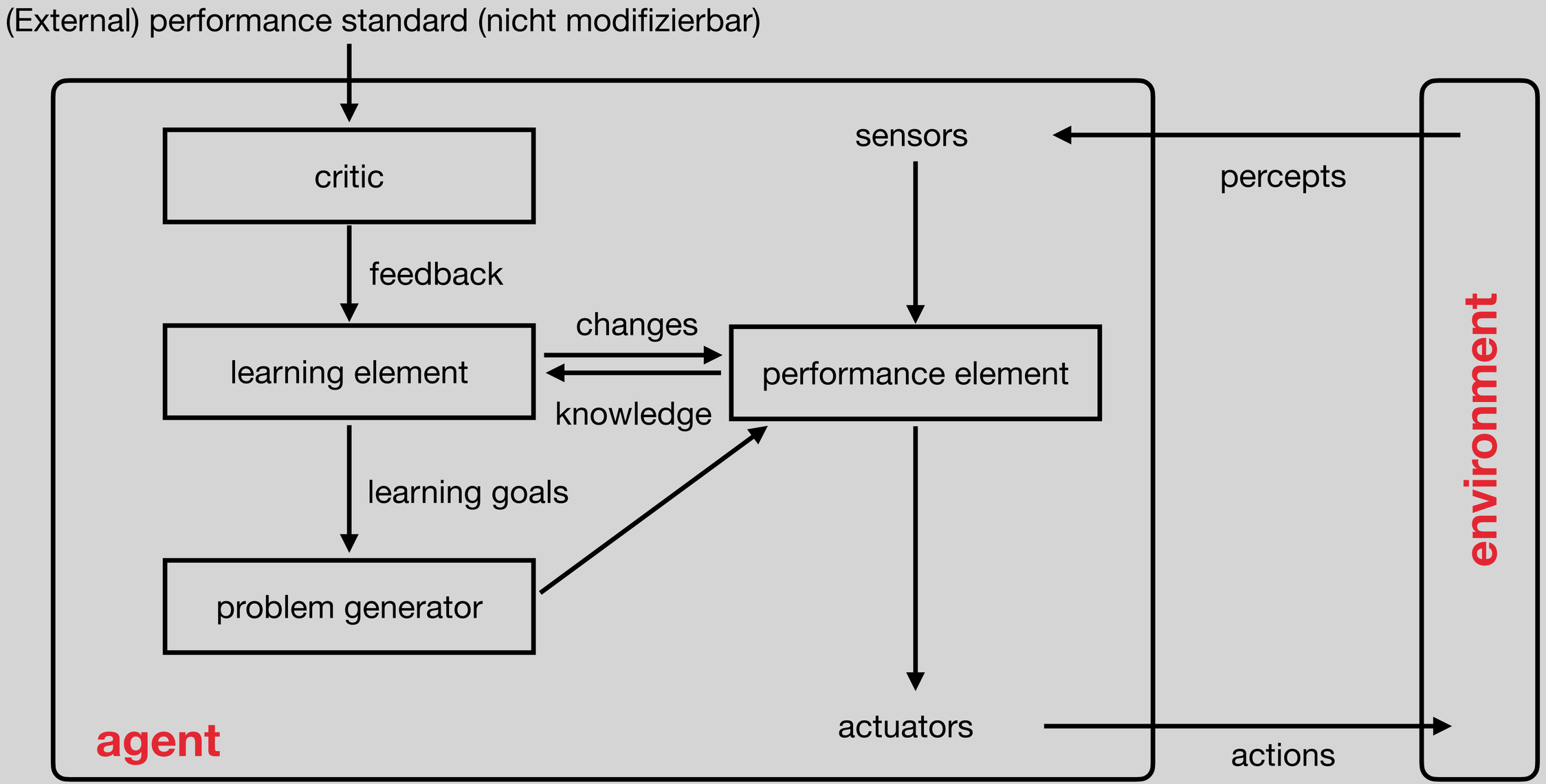
Mehr oder weniger handelt es sich in der Arbeit von Turing um die Geburt der Idee zum **maschinellen Lernen** (machine learning).

Alle zuvor besprochenen Agenten (also einfache & Modell-basierte Reflex Agenten, Ziel-basierte Agenten und Utilitäts-basierte Agenten) lassen sich als Lernagenten realisieren. In nahezu jedem Gebiet von AI sind Lernagenten aktuell der de-facto Standard um State-of-the-Art Systeme umzusetzen.

Lernen hat zudem den Vorteil, dass Agenten in initial unbekannten Umgebungen operieren können und über die Zeit immer kompetenter werden.

Die Komponenten eines Lernagenten sind auf der folgenden Folie dargestellt.

Struktur von Agenten



Struktur von Agenten

Das **Lernelement** (learning element) ist verantwortlich, um Verbesserungen zu machen; das **Performanzelement** (performance element) wählt schlussendlich Handlungen aus. Im Wesentlichen subsummiert das Performanzelement alle Komponenten unserer bisherigen Agenten.

Das Lernelement nutzt Feedback vom **Bewerter** (critic) über die Performanz des Agenten um das Performanzelement zu modifizieren (alle Elemente in den Diagrammen zu den vorherigen Agenten können modifiziert werden).

Struktur von Agenten

Der Bewerter nutzt den externen **Performanzstandard** (performance standard; **nicht modifizierbar**), um dieses Feedback zu geben. Die Sensorik kann hierzu nicht dienen (z.B. die Stellung Schachmatt im Schach soll zwar über Sensorik erkannt, aber nicht bewertet werden).

Beim Performanzstandard handelt es sich um ein Element, das Teile einer Sequenz an Wahrnehmungsobjekten mit Strafe (penalty) od. Belohnung (reward) bewertet.

Der **Problemgenerator** (problem generator) **schlägt Handlungen vor**, um neue informative Erfahrungen zu machen. Diese sind oft suboptimal, erlauben es jedoch, auf längere Sicht gesehen, bessere Handlungen zu finden.

Repräsentation der Umgebung

Bislang haben wir verschiedene Komponenten betrachtet, welche im Wesentlichen folgende Fragen (aus Sicht des Agenten) beantworten:

- Wie ist der aktuelle Zustand meiner Umgebung?
- Welche Handlung soll ich durchführen?
- Welche Konsequenzen haben meine Handlungen?

Ein wichtiger Punkt in diesem Kontext ist die Art und Weise, wie unsere verschiedenen Komponenten die Umgebung des Agenten **repräsentieren**.

Repräsentation der Umgebung

Wir unterscheiden grob zwischen

- atomaren,
- faktorisierten und
- strukturierten

Repräsentationen (gelistet in der Reihenfolge aufsteigender Ausdrucksstärke).

Repräsentation der Umgebung

In einer **atomaren Repräsentation** ist der Zustand der Umgebung nicht aufteilbar—der Zustand hat keine interne Struktur.

Beispiel: Wir werden später ein Routensuchproblem sehen, bei dem der Zustand der Umgebung durch den Namen einer Stadt repräsentiert ist. Dieser atomare Zustand unterscheidet sich von anderen atomaren Zuständen, ist aber quasi als “Black Box” nicht aufteilbar.

Repräsentation der Umgebung

In einer **faktorisierten Repräsentation** ist der Zustand der Umgebung in eine fixe Menge an Variablen und Attributen aufgeteilt. Diese Variablen und Attribute können Werte annehmen.

Beispiel: Im zuvor genannten Routensuchproblem könnte man den Zustand der Umgebung anhand der Attribute “Treibstoffstand”, “GPS Koordinaten”, “Ölwarnleuchte an/aus” charakterisieren.

Repräsentation der Umgebung

In einer **strukturierten Repräsentation** ist der Zustand der Umgebung charakterisiert durch Dinge/Objekte und deren Relation untereinander; also nicht mehr alleine durch Variablen und Attribute mit Werten.

Vieles, was wir in natürlicher Sprache ausdrücken, handelt von Objekten und deren Beziehungen untereinander/miteinander.

Repräsentation der Umgebung

Von atomar hin zu faktorisiert und strukturiert steigt die **Ausdrucksstärke** (expressiveness) der Repräsentation.

In vielen Fällen kann man mit einer ausdrucksstärkeren Repräsentation Dinge deutlich knapper ausdrücken. Beispielsweise sind die Regeln von Schach auf ein paar Seiten in einer strukturierten Repräsentationssprache (z.B. Logik erster Stufe) beschreibbar, man benötigt aber $\approx 10^{40}$ Seiten würde man das Gleiche in einer atomaren Repräsentationssprache (z.B. in der eines endlichen Automaten) versuchen.

Problemlösen mittels Suche

Kapitel 3 (RN)

Problemlösen mittels Suche
—
in aller Kürze

Ein Agent kann **vorausschauen**,
um eine **Sequenz an Handlungen** zu finden,
die letztlich zur Zielerreichung führen.

Problemlöseagenten

- Ist die richtige nächste Handlung nicht unmittelbar ersichtlich
  → Agent muss **vorausplanen**
  d.h. **Sequenz** von potentiellen Handlungen betrachten, die zum Ziel führen (**Pfad**)

- Solche Agenten nennt man **Problemlöseagenten** (problem-solving agents)
- zugrundeliegende Prozess = **Suche**

- Problemlöseagenten nutzen **atomare** Repräsentationen (atomic representations)
  - Zustände (states) der Welt werden in ihrer Gesamtheit betrachtet
  - den “Problemlösealgorithmen” ist keine innere Struktur der Welt zugänglich

Agenten,
die *faktorisierte* oder
strukturierte Repräsentationen von
Zuständen nutzen,
nennt man **Planungsagenten**
(planning agents).

Beispiel: Tour durch Rumänien

Zustand (state) = **Arad** ■
Arad

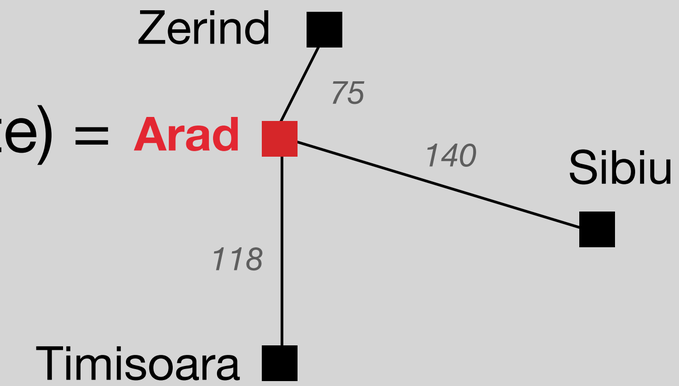
Bucharest ■

Ziel = Bucharest

Zielformulierung =
“reaching Bucharest”

Beispiel: Tour durch Rumänien

Zustand (state) =
Arad



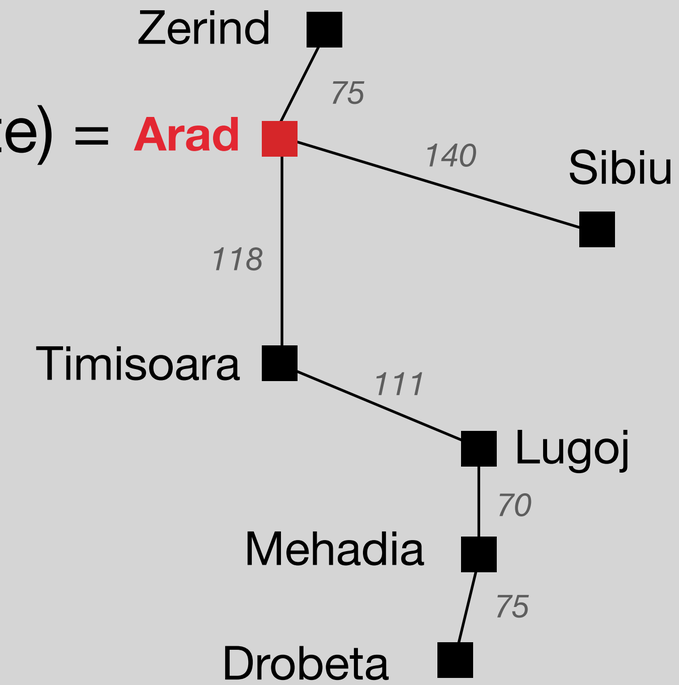
Bucharest

Ziel = Bucharest

Zielformulierung =
“reaching Bucharest”

Beispiel: Tour durch Rumänien

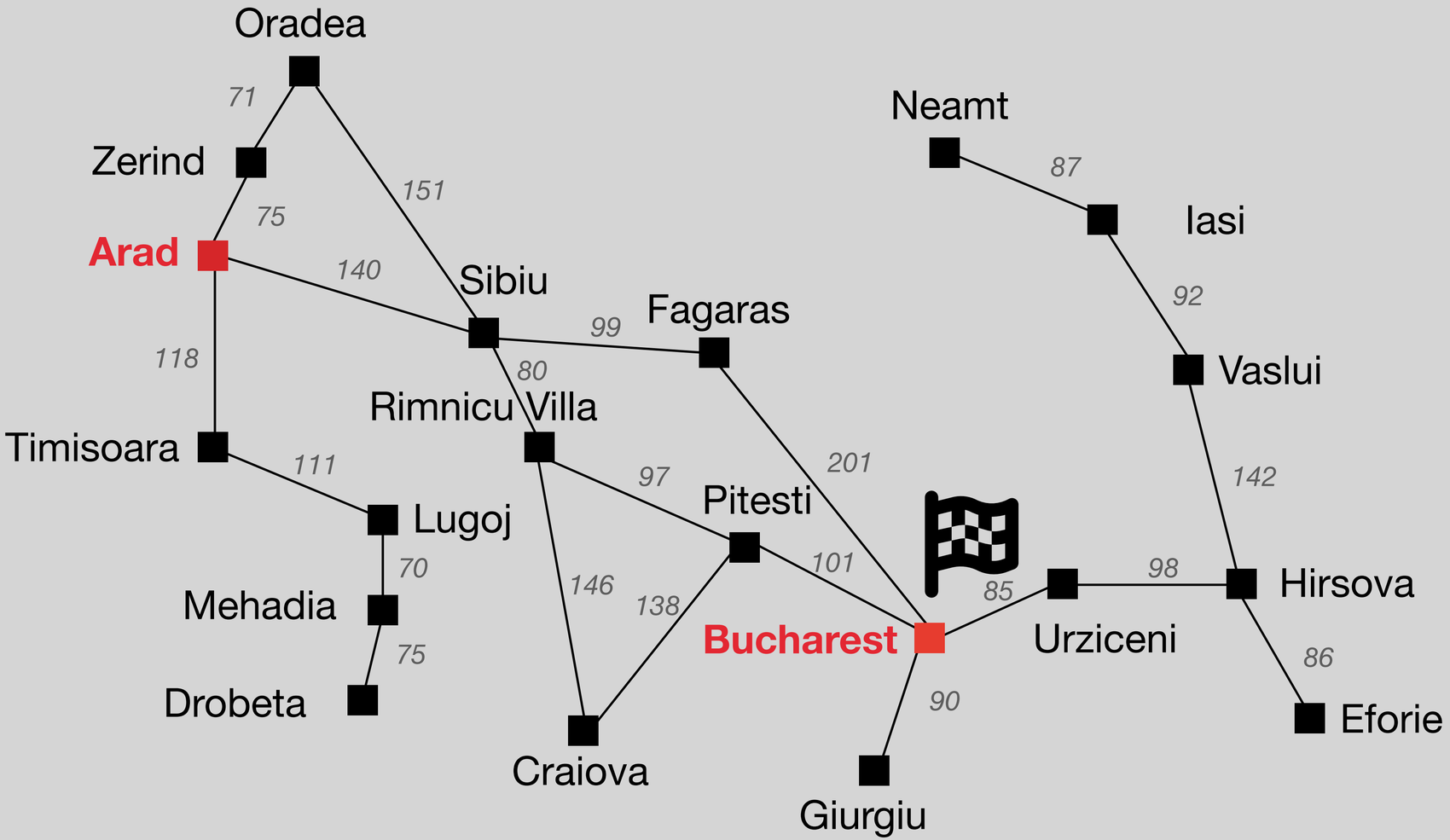
Zustand (state) =
Arad



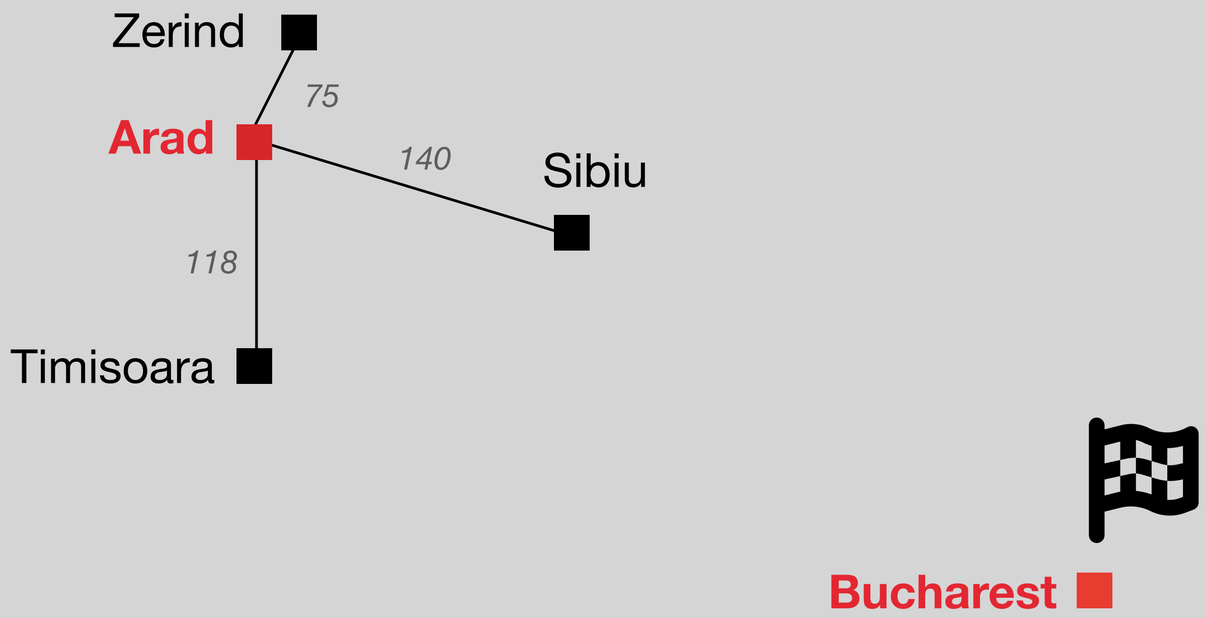
Ziel = Bucharest

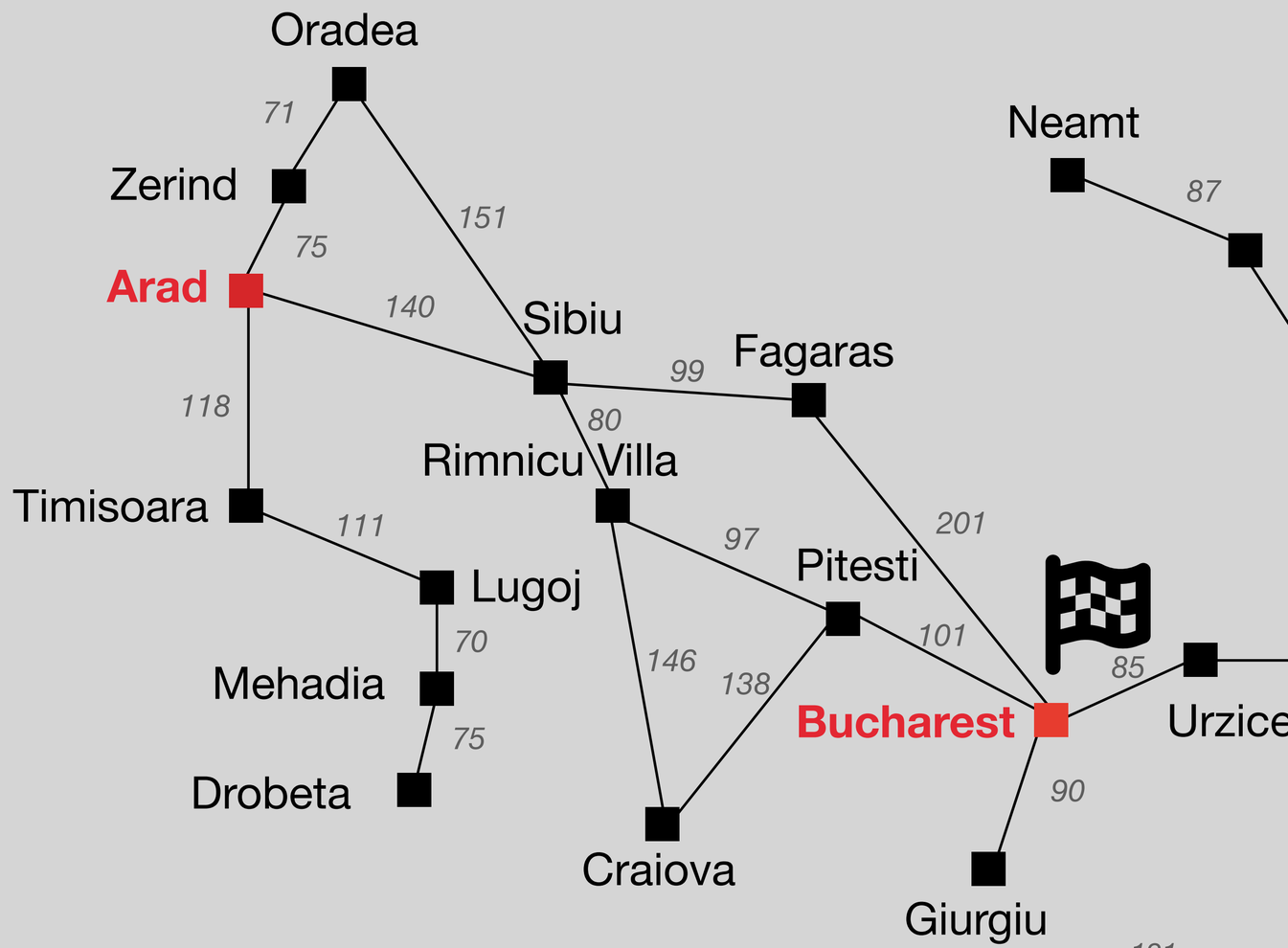
Zielformulierung =
“reaching Bucharest”

Beispiel: Tour durch Rumänien



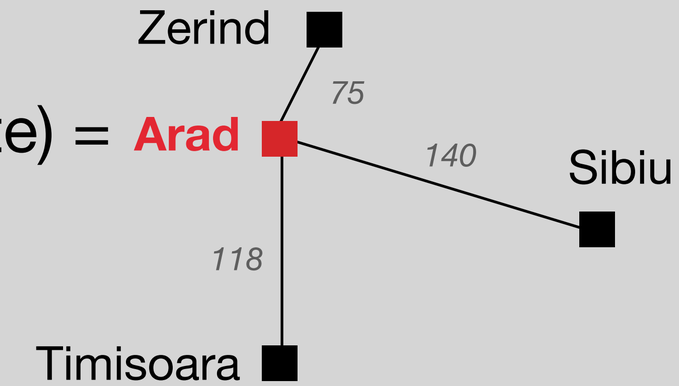
Unterscheidung: Problemlöseagenten bei unbekannter oder bekannter Aufgabenumgebung





Aufgabenumgebung unbekannt

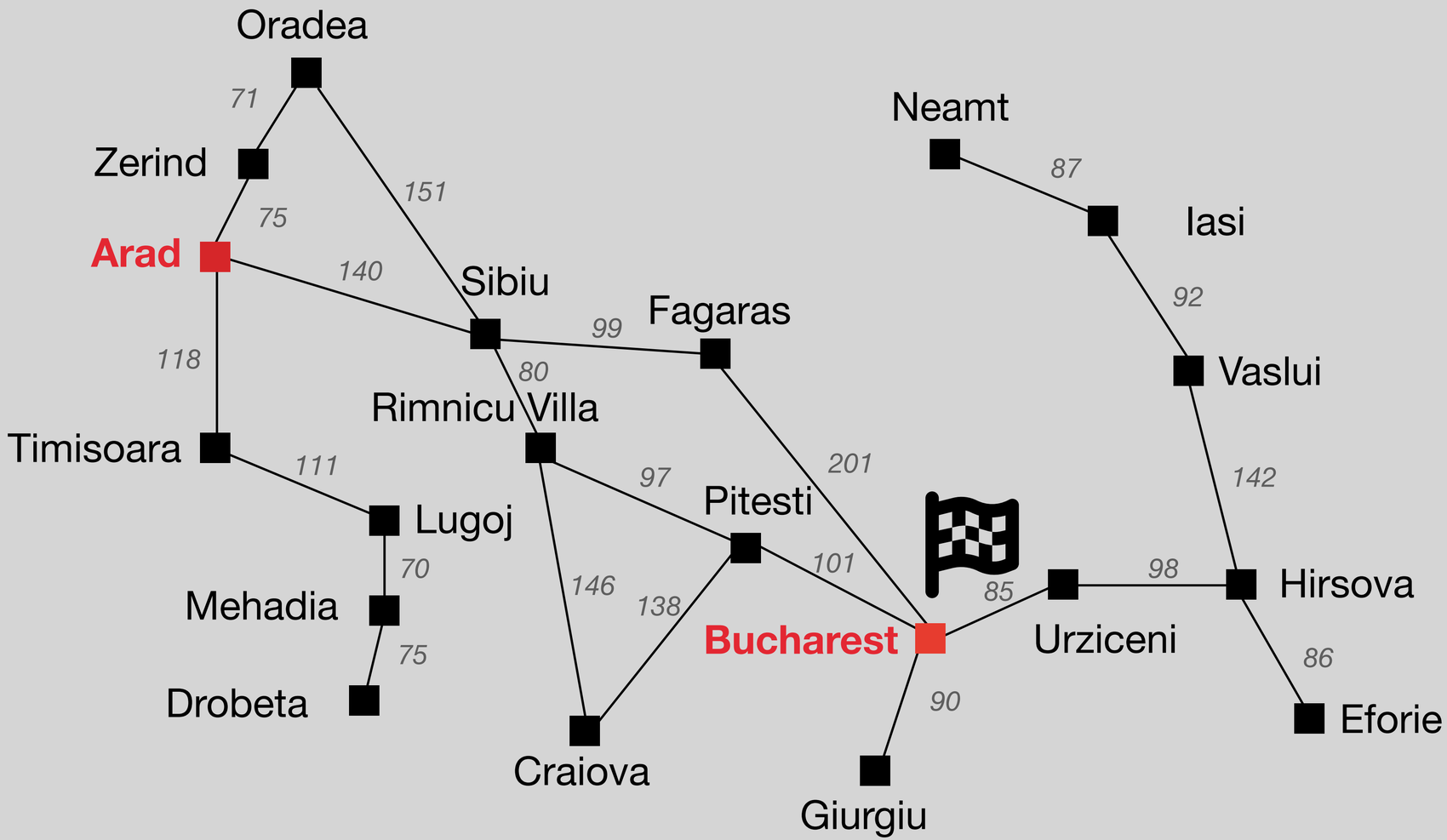
Zustand (state) =
Arad



Agent kann nur
zufällig
einen der drei Wege weg von Arad wählen
(nach Sibiu, Timisoara od. Zerind)

Aufgabenumgebung bekannt

hier: Karte bekannt



Agent kann den **4 Stufen** des Problemlöseprozesses folgen:

1. Formulierung des Ziels (goal formulation)
2. Formulierung des Problems (problem formulation)
3. Suche (search)
4. Durchführung (execution)

1. Formulierung des Ziels (goal formulation)

- Ziel ist Bucharest
- Zielformulierung = “reaching Bucharest”
- Ziel schränkt die zu berücksichtigenden Handlungen ein.

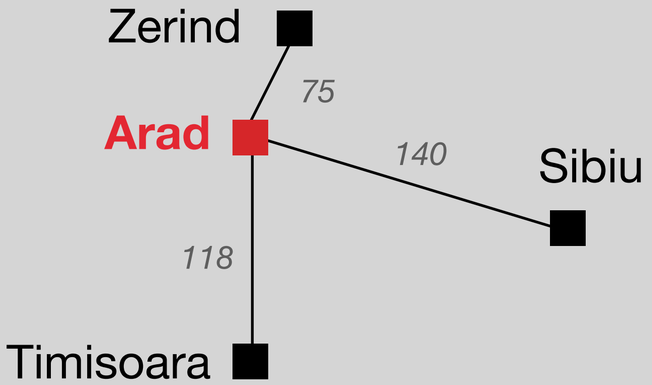
Ziel = Bucharest

Zielformulierung =
“reaching Bucharest”

2. Formulierung des Problems (problem formulation)

- Agent erstellt **abstraktes Modell** der relevanten Teile seiner Umgebung
  → d.h. Zustände u. Handlungen, die notwendig sind, um das Ziel zu erreichen

- *ein mögliches* gutes Modell für unseren Agenten:
  - nur Handlungen betrachten, die den Agenten von einer Stadt in eine benachbarte Stadt bringen
  - Der einzige Teil (hier: Info) des Zustands, der sich nach einer Handlung ändert, ist die aktuelle Stadt

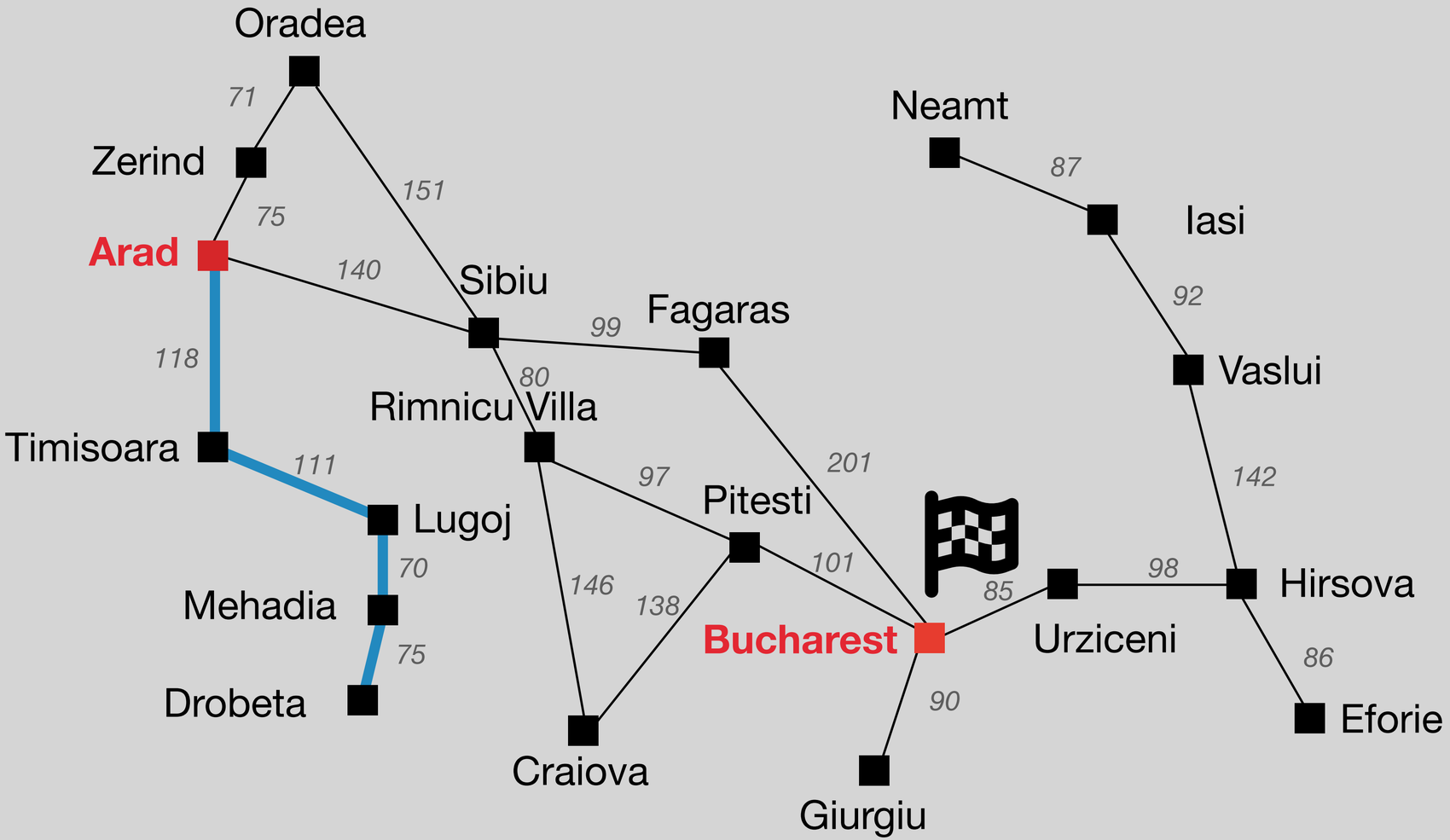


z.B.
state: Arad
state: Zerind
state: Timisoara
state: Sibiu

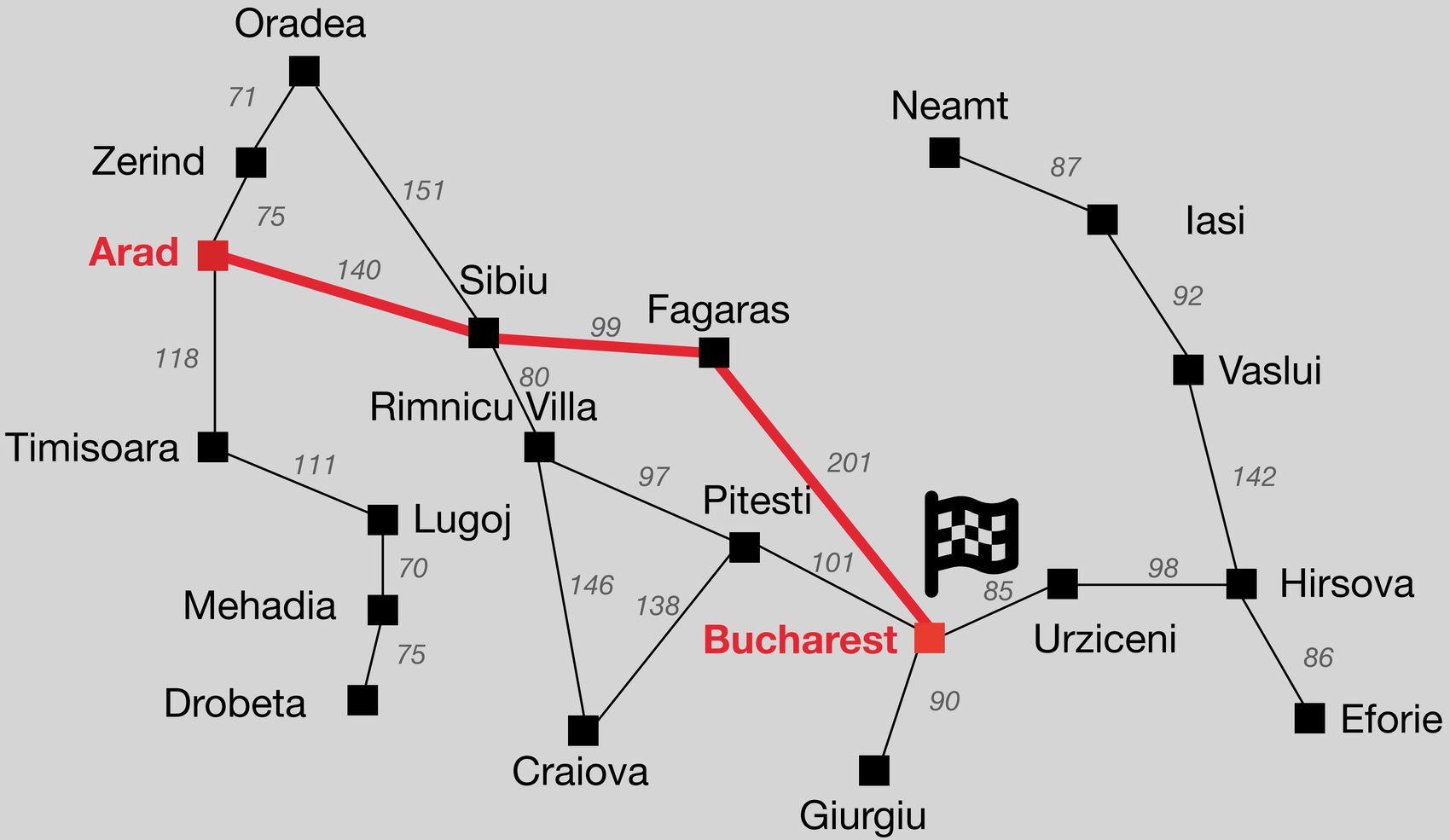
3. Suche (search)

- Bevor der Agent eine Handlung ausführt:
  - Agent **simuliert** Handlungssequenz in seinem Modell, bis er eine Handlungssequenz findet, mit der das Ziel erreicht wird
- Sequenz zur Zielerreichung = **Lösung** (solution)
- i.d.R. Simulation vieler Sequenzen notwendig:
  - Agent muss ev. mehrere Sequenzen simulieren, die das Ziel nicht erreichen, um letztlich eine Lösung finden
  - ev. auch Resultat: keine Lösung möglich

Beispiel einer Sequenz, die nicht zum Ziel führt



Lösung zur Zielerreichung



4. Durchführung (execution)

- Agent führt die Handlungen der gefundenen Lösung durch, Schritt für Schritt.
- In voll beobachtbaren, deterministischen, bekannten Aufgabenumgebungen
  → Lösung zu jedem Problem ist **fixe** **Handlungssequenz**
  - Wahrnehmungsobjekte können bei der Durchführung der gefundenen Lösung ignoriert werden
  - →**Open-loop** System, da es keine “(Feedback)Schleife” zwischen Agenten und Umgebung mehr gibt.

Begriff aus der *Kontrolltheorie*

- In teilweise beobachtbaren oder in nicht-deterministischen Umgebungen müsste eine Lösung ja Handlungsvorschläge auf Basis aktueller Wahrnehmungsobjekte beinhalten (Verzweigungsstrategie; closed-loop).

z.B. Agent plant von Arad nach Sibiu zu fahren; aber Notfallsplan, falls er versehentlich in Zerind ankommt od. Schild “Straße gesperrt” vorfindet.

Formale Beschreibung eines Suchproblems

- **Menge an Umgebungszuständen** (state space)

  z.B. S = {Arad, Sibiu, Timisoara, Zerind, …}

- **Initialzustand** (initial state)

- **Menge an Zielzuständen** (goal states)

  z.B. G = {Bucharest}

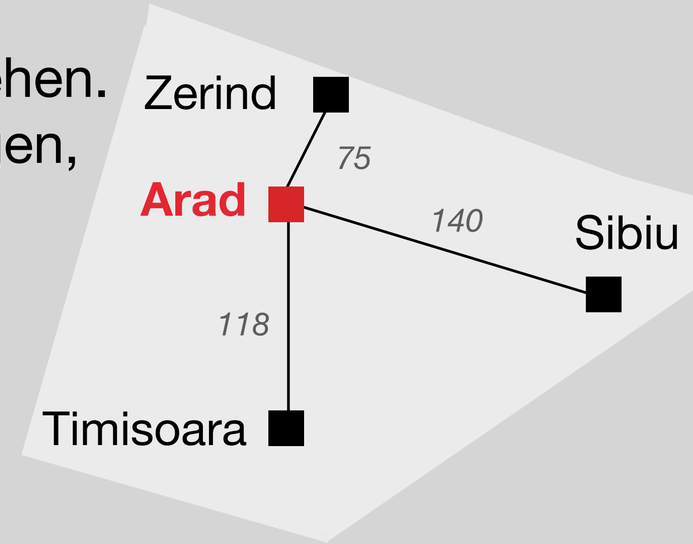
es kann auch mehrere Zielzustände geben

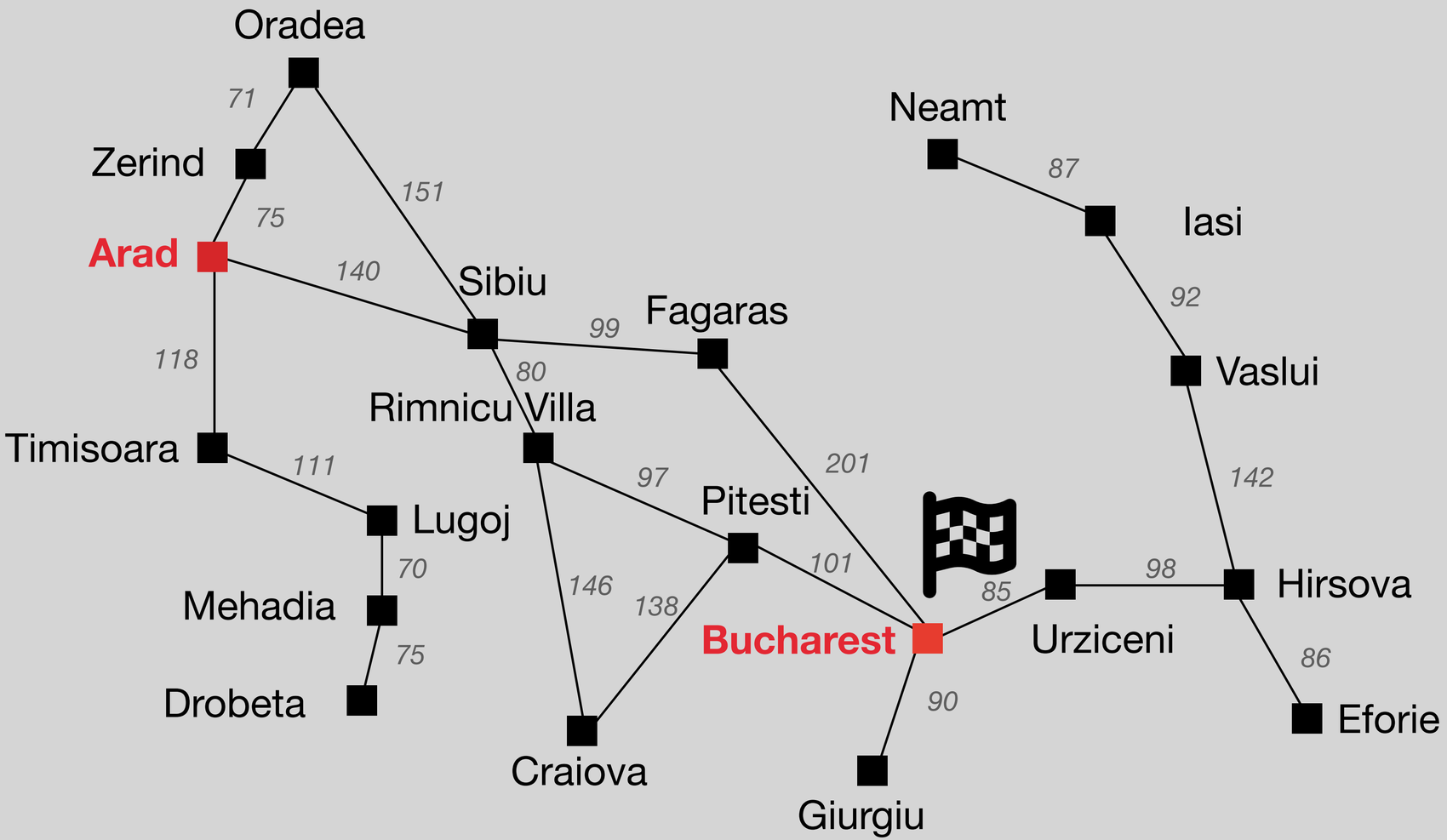
- **Menge an Handlungen** (actions), die dem Agenten zur Verfügung stehen. Bei gegebenen Zustand s, liefert ACTIONS(s) die Menge an Handlungen, die in Zustand s durchgeführt werden können.

  z.B. ACTIONS(Arad) = {ToSibiu, ToTimisoara, ToZerind}

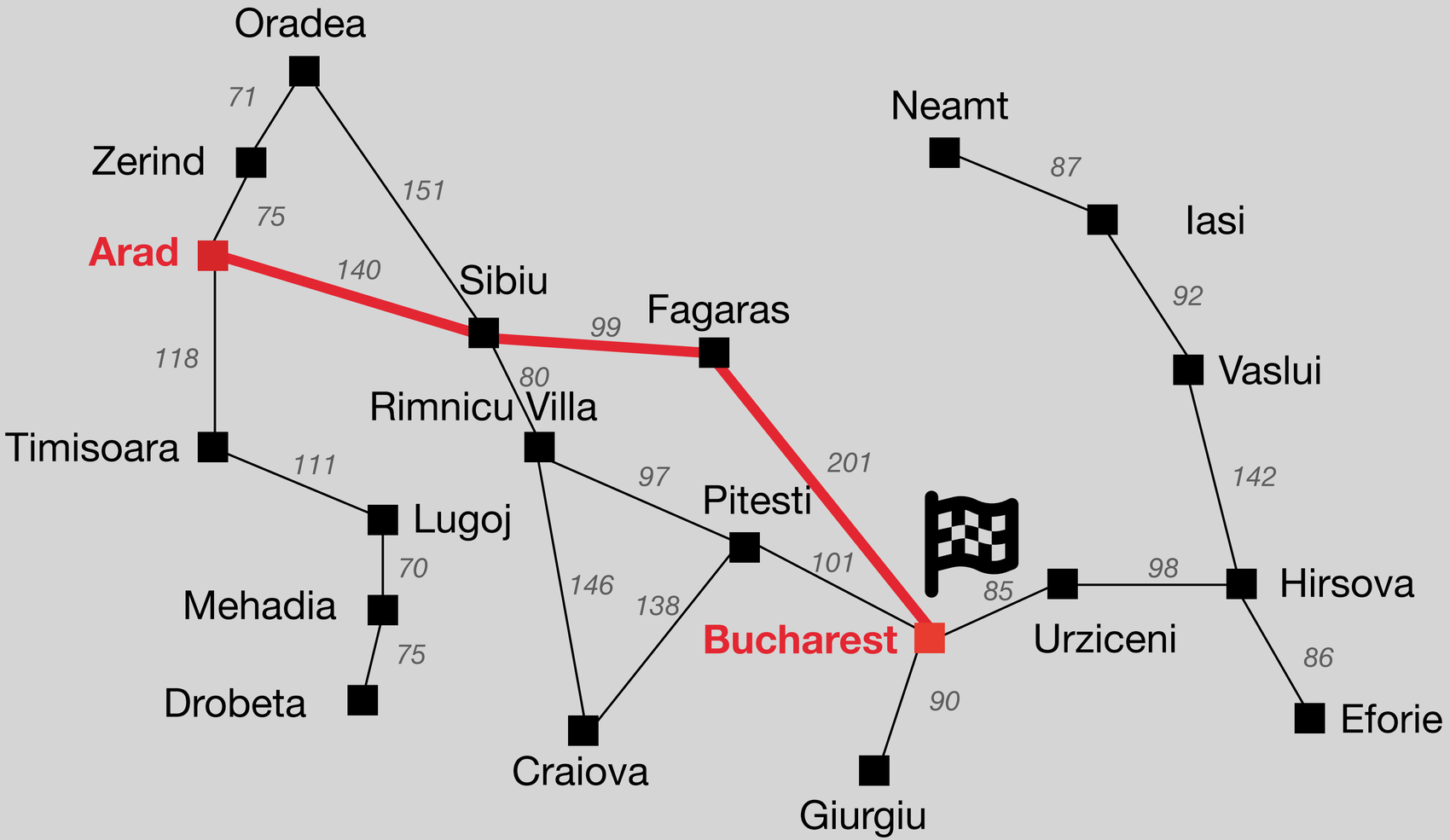
- **Übergangsmodell** (transition model), das beschreibt, was in einem Zustand bei gegebener Handlung passiert

  z.B. RESULT(Arad, ToZerind) = Zerind

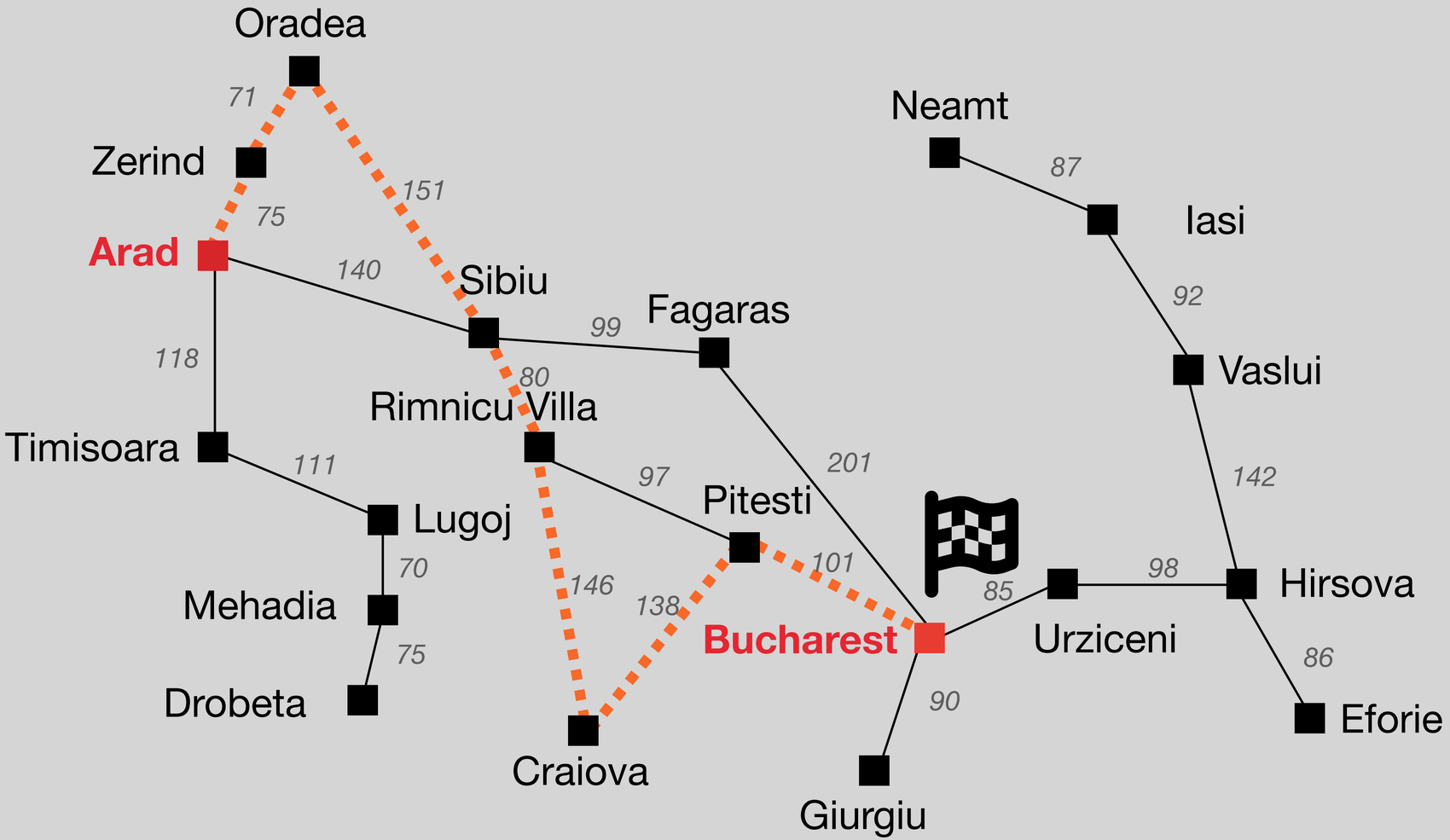



Oradea
71
Neamt
87
Zerind
151
Iasi
75
Arad
140
Sibiu
92
99
Fagaras
118
80
Vaslui
Rimnicu Villa
Timisoara
111
97
201
142
Pitesti
Lugoj
70
101
98
Hirsova
146
85
138
Mehadia
Bucharest
Urziceni
75
86
Drobeta
90
Eforie
Craiova
Giurgiu

Lösung zur Zielerreichung



Lösung zur Zielerreichung



Handlungs-Kostenfunktion (action cost function)

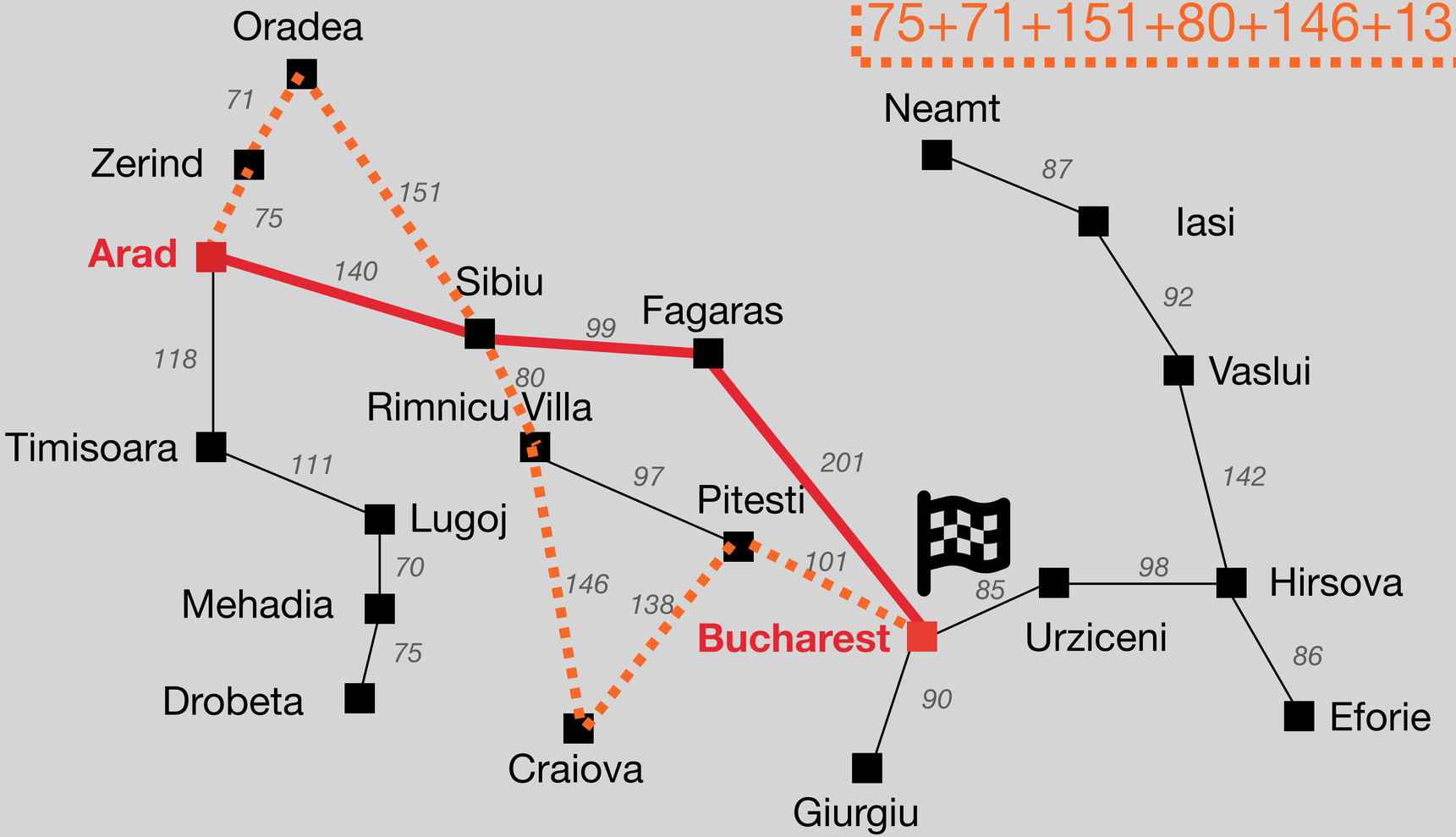
- **Handlungs-Kostenfunktion** (action cost function): ACTION-COST(s, a, s') gibt uns die “Kosten” die Durchführung der Handlung a in Zustand s durchzuführen und Erreichung des Zustands s'.
  - In unserem Beispiel → Kostenfunktion z.B. Distanz in Kilometern oder benötigte Zeit.

- Eine Handlungssequenz nennt man auch **Pfad** (path).
- Angenommen die Kosten sind additiv, dann ist eine **optimale Lösung** (optimal solution) jene Lösung mit der niedrigsten Summe an Kosten entlang eines Pfades unter allen möglichen Lösungen.

Optimale Lösung

140+99+201=440

75+71+151+80+146+138+101=762



Problemformulierung als Abstraktion

- Problemformulierung “nach Bucharest zu kommen” ist ein Modell
  → eine **Abstraktion** der Wirklichkeit
  - Die atomare Zustandsbeschreibung von beispielsweise Arad umfasst nicht alle Details einer tatsächlichen Reise (z.B. Wetter, Tankfüllstand etc.).
  - Für unser Beispiel sind alle zusätzlichen Details jedoch irrelevant.
  - Das “Weglassen” od. “Reduzieren” nicht notwendiger Details nennt man **Abstraktion**.
- Eine “gute” Problemformulierung hat **den gerade “richtigen” Detailgrad**.
  (→ Oft die schwierigste Aufgabe!)
- Eine Abstraktion ist dann **nützlich** (useful), wenn das Durchführen der Handlungen in der Lösung einfacher ist als im Originalproblem.

Problemformulierung

Eine Abstraktion ist **valide** (valid),
wenn eine Lösung in der Abstraktion des Problems in eine Lösung in der detaillierteren Welt übergeführt werden kann.

Um ein konkretes Problem abstrahiert zu betrachten, versuchen wir:

(1) Details soweit wie möglich zu entfernen,

(2) Handlungen einfach durchführbar zu halten und

(3) die Validität nicht zu beeinträchtigen.

Problem-Beispiele

Wir unterscheiden zwischen **standardisierten** und **“real-world”** Problemen.

Standardisierte Probleme dienen dazu, verschiedene Problemlösemethoden zu testen und zu evaluieren. “Real-world” Probleme sind jene die auch tatsächlich in der Realität vorkommen (meist individuelle Probleme mit bestimmten Sensoren, etc.).

Problem-Beispiele

Standardisierte Probleme

Grid-world: 2-dimensionales Array an quadratischen Kacheln; ein Agent kann sich zwischen diesen Kacheln bewegen, typischerweise horizontal od. vertikal von einer Kachel zu einer (hindernisfreien) anderen Kachel. In Kacheln können sich Objekte befinden, die der Agent bewegen od. verschieben kann (od. auf die er in sonnst einer Art und Weise einwirken kann). Wände od. Hindernisse verhindern, dass sich ein Agent von einer Kachel zur Nächsten bewegen kann.

Ein klassisches Beispiel ist die Staubsaugerwelt von zuvor.

Problem-Beispiele

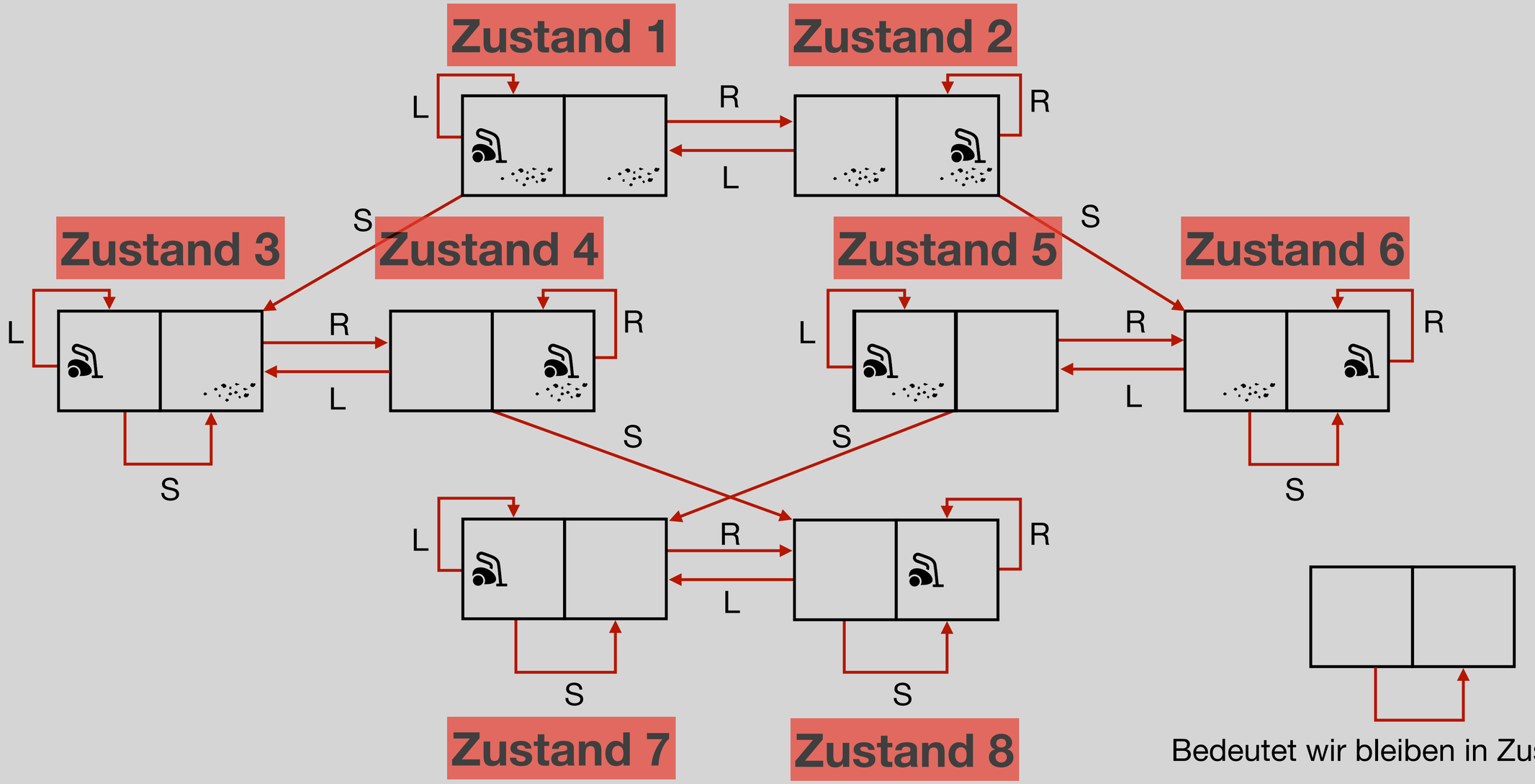
Formuliert als ein Grid-world problem, haben wir für unser Staubsaugerwelt Beispiel folgende Beschreibung:

Umgebungszustände (states): ein Zustand beschreibt welche Objekte sich in welchen Kacheln befinden; Objekte sind hier der Agent und Schmutz. Bei zwei Kacheln (A, B) kann der Agent entweder in A od. B sein und A bzw. B können schmutzig od. sauber sein. Demnach haben wir

$$2 \cdot 2^2 = 2 \cdot 2 \cdot 2 = 2^3 = 8$$

Zustände (siehe Illustration auf nächster Folie).

Problem-Beispiele



Problem-Beispiele

- **Initialzustand** (initial state): Jeder Zustand kann Initialzustand sein
- **Handlungen** (actions): “saugen”, “bewege-links”, “bewege-rechts”
- **Übergangsmodell** (transition model): “saugen” entfernt Schmutz, etc.
- **Zielzustände** (goal states): Zustände in denen alle Kacheln sauber sind
- **Handlungs-Kostenfunktion** (action costs): jede Handlung kostet +1.

Suchalgorithmen

Wir betrachten Algorithmen die einen **Suchbaum** über dem Zustandsraum (state space) aufbauen.

Wir versuchen einen **Pfad** in diesem Suchbaum zu finden der, ausgehend von einem Startknoten, unser Ziel erreicht.

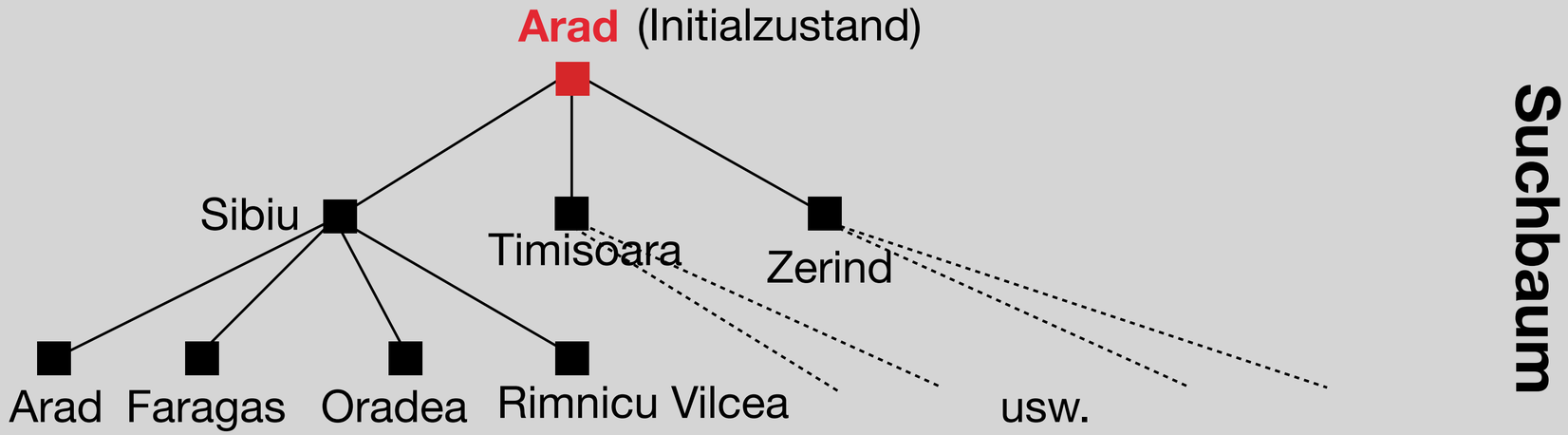
Im Suchbaum gibt es **Knoten** und **Kanten**:

- Knoten im Baum = entsprechen Zuständen im Zustandsraum (z.B. Arad)
- Kanten im Baum = entsprechen Handlungen (z.B. ToZerind, etc.)

Der Wurzelknoten (root note) im Baum ist unser **Initialzustand**.

Suchalgorithmen

Der Suchbaum (im Rumänien Bsp.) sieht folgendermaßen aus (nur Ausschnitt).



Wurde eine konkrete Handlung durchgeführt (z.B. ToSibu), werden wir den Zielknoten als **erreicht** bezeichnen.

Werden nur die potentiellen Handlungen betrachtet und wir kennen daher die potentiellen Zielknoten, nennen wir den Knoten **erweitert** (siehe nächste Folie).